Trade Desk: the design

The trunk and every document below it, in the order of the tree.

1. **Trade Desk: the architecture** · `ARCHITECTURE.md`
2. **The box** · `design/box/BOX.md`
3. **The front door** · `design/front-door/FRONT-DOOR.md`
4. **The desk** · `design/desk/DESK.md`
5. **Files on the box** · `design/files/FILES.md`
6. **qp: charts, strategies, backtests, alerts** · `design/quant/QUANT.md`
7. **The research worker** · `design/research/RESEARCH-WORKER.md`
8. **The journal** · `design/journal/JOURNAL.md`
9. **Data sources** · `design/outside/SOURCES.md`
10. **The shared record** · `design/outside/SHARED-RECORD.md`
11. **Backups** · `design/outside/BACKUPS.md`
12. **Alerts and the health page** · `design/outside/ALERTS.md`
13. **From today's desk to the design** · `design/MIGRATION.md`

Trade Desk: the architecture

This is the top of the design. It says what the Trade Desk is, shows the whole of it in one picture, and explains each block of that picture in a few lines.

How to read the design. It is a tree. Each block in the picture has its own document one level down, and each part of a block has its own document below that. Every document names the one above it in its first line. Read this one first, then go down only into the branch you need.

Contents

1. What the desk is
2. What shapes the design
3. The whole desk, in one picture
4. The blocks
5. How the blocks share data
6. A trading day
7. Rules that apply everywhere
8. Running it
9. Where the build stands, and the way there
10. Decisions
11. Risks
12. The document tree
13. Words used here

---

1. What the desk is

The Trade Desk is a private research desk for one day trader. Each trading morning it finds the stocks worth looking at, using several screening methods side by side. It shows everything worth knowing about each of those stocks on one card. And it keeps an honest record of what each method found and how those stocks then moved, so the trader can learn which methods are worth trading.

It also charts and backtests trading rules, alerts the trader when a saved rule fires, and reviews the trader's real trades. **It never places an order.**

What it does

Job	The question it answers
Find	which stocks are worth looking at right now, by each of my methods?
Know	what is there to know about this stock today, on one card?
Read the market	should I be trading at all today, and where is the strength?
Learn	which methods find stocks that move?
Test	if I had traded this rule on what my methods found, what would have happened?
Review	how did my real trades go?

How it learns

The desk is a research instrument before it is a trading screen:

- **Each method is a tool.** A tool is one way of finding stocks, run as an experiment, with its own history. Tools never share results.
- **Each tool records what it found, once a day,** at a fixed minute in the first hour of trading.
- **After the close, the desk records how far each of those stocks moved.**
- **Over weeks, that record shows which tools find stocks that move,** against a plain benchmark tool that every other must beat.
- **Facts about the world are shared; results are not.** A fact (a stock's news, the market's trend) can label a card, but it never changes which stocks a tool finds.

How the record is taken and measured is described two levels down, in the documents of a running tool.

---

2. What shapes the design

What it must live with

Constraint	What follows
one trader	one login; no accounts or roles; simple beats scalable
one small server with 1 GB of memory	every program has a memory budget, and a tool must cost little
free data	free and public sources, two of them unofficial; each can change or fail without notice
New York hours	the desk works from 04:00 to the close on trading days, and the first hour after the open matters most
no orders	the desk finds, explains, measures and records; it never trades

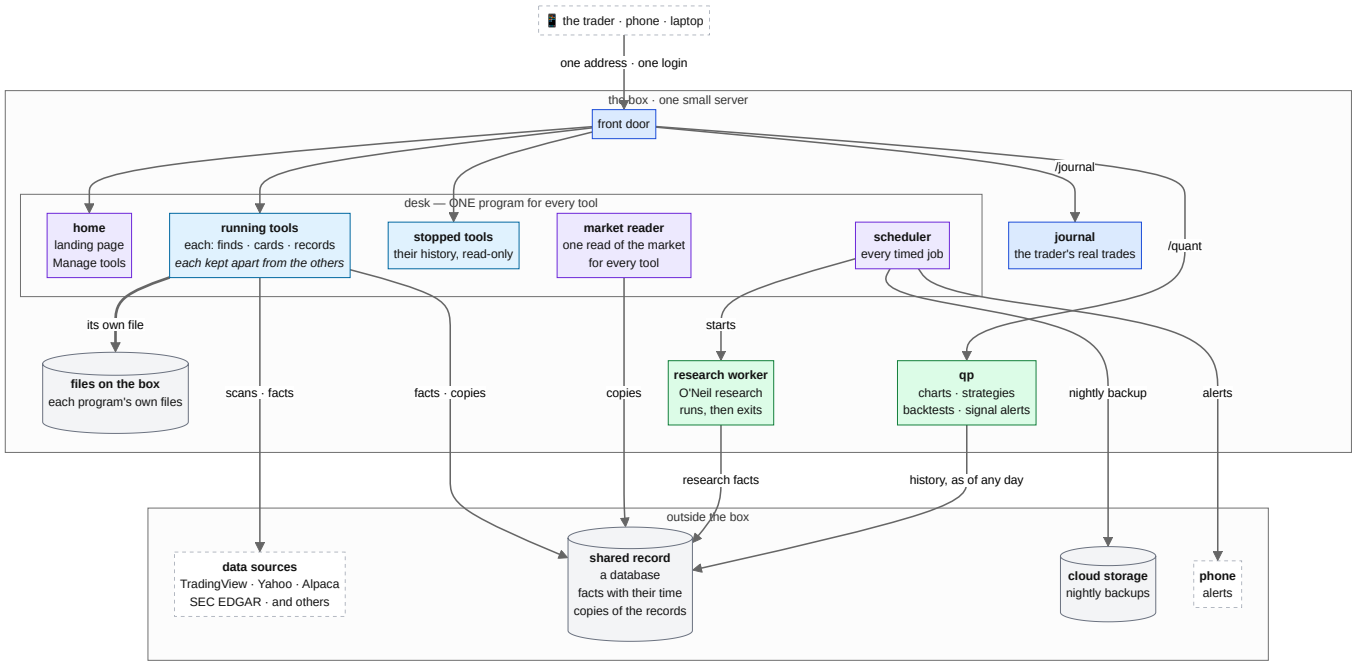
What it must achieve, in order

When two goals pull against each other, the higher one wins.

1. **A true record.** What a tool records holds exactly what was known at that minute: nothing later leaks in, and nothing missing is guessed.
2. **No silent failure.** Anything that fails, or does not run, shows on one page and reaches the phone.
3. **Ready every morning.** A program that crashes restarts in seconds and loses nothing.
4. **Fits the server.** Every program stays inside its memory budget.
5. **One person can run it.** One command to deploy, one to restore, one page for its health.
6. **Every number explains itself.** Its source, its time and its meaning are one tap away.

---

3. The whole desk, in one picture



Some lines are left out to keep the picture readable: the scheduler also starts the work of qp and the journal; qp and the journal keep their own files; and the backup copies the shared record as well as the files.

Block	What it does	Its document
the box	the one server everything runs on	design/box/BOX.md
front door	the one address and the one login	design/front-door/FRONT-DOOR.md
desk	one program that runs every tool	design/desk/DESK.md
files on the box	where each program keeps what it records	design/files/FILES.md
qp	charts, strategies, backtests, signal alerts	design/quant/QUANT.md
research worker	the slow, heavy research: the O'Neil method	design/research/RESEARCH-WORKER.md
journal	the trader's real trades, after the fact	design/journal/JOURNAL.md
data sources	everything the desk reads from outside	design/outside/SOURCES.md
shared record	the facts and copies more than one program needs	design/outside/SHARED-RECORD.md
backups	a copy of everything, off the box, every night	design/outside/BACKUPS.md

Block	What it does	Its document
alerts	what reaches the phone, and the health page	design/outside/ALERTS.md

How today's desk becomes this one: design/MIGRATION.md.

---

4. The blocks

4.1 The front door

The trader reaches the whole desk at **one address**, from a phone or a laptop, with **one login**. The front door sends each request to the right program and keeps everything else closed to the outside. It holds nothing and computes nothing: if it stops, the desk keeps working, and only the pages are out of reach.

4.2 The desk

One program runs every tool. For each tool it finds the stocks, builds their cards, keeps the day's list, records what the tool found, and serves the tool's pages.

Its parts:

- **Home:** the landing page, the lists across all tools, and the page that adds, starts, stops and edits tools. Home belongs to the desk itself, so any tool can be stopped.
- **Running tools:** each one kept apart from the others inside the program, so no tool can see or change another's results.
- **Stopped tools:** what each one ever recorded, still readable.
- **The market reader:** one read of the market for every tool, kept with its history.
- **The scheduler:** the only clock of the whole desk. It runs every timed job, including the work of the other programs, and records every run.

Why one program. A tool costs the desk a little memory instead of a whole program; there is one clock, so nothing runs twice; and nothing a page shows has to be copied between programs (§10).

4.3 Files on the box

Each program keeps what it records in **its own files on the box**, and every file has **exactly one program that writes it**. Each tool's results are kept in that tool's own file, so the desk works without the network and the tools stay apart.

4.4 qp

The chart and backtest program, kept as it is because it is the best-built part of the old desk:

- a library of indicators, each checked against TradingView;
- the chart;
- a builder for trading rules;
- the backtest of a rule on the stocks the tools found, on the days they found them;
- signal alerts when a saved rule fires on today's stocks, on its page and on the phone.

It has no clock of its own: the desk's scheduler starts its alerts. It reads the tools' history from the shared record.

4.5 The research worker

A job the scheduler starts, which does the slow, heavy research of the O'Neil method (earnings, relative strength, industry groups, the market's trend), writes its results with their dates into the shared record, and exits. It is the only writer of that research, and no page ever waits for it.

4.6 The journal

The trader's real trades, after the fact: from the broker's records, from uploaded files, or typed in. It turns them into complete trades, measures them, and keeps the trader's notes. It has its own program and file, because it shares nothing live with the tools.

4.7 Outside the box

Block	What it is
data sources	TradingView's scanner, Yahoo, Alpaca, SEC EDGAR and others. Each is reached through one piece of code that knows its limits, and that says "unavailable, because …" rather than guessing
shared record	a database for what more than one program needs: facts about the world with the time they became known, the O'Neil research, and a copy of every tool's records
backups	every night, a copy of every file and of the shared record goes to cloud storage, and is checked by restoring it. One command puts a night back

Block	What it is
alerts	anything that fails reaches the phone once, and its recovery once. A check from outside the box says when the desk itself has stopped. One page shows the health of everything

---

5. How the blocks share data

5.1 Three kinds of data

Kind	Example	Rule
facts about the world	a stock's short interest; the index's close; a company's quarterly earnings	shared by every tool; stored once, with the time it became known and its source
results of a method	what a tool found this morning, and how those stocks moved	belong to one tool; never shared, never mixed
the trader's records	strategies, backtests, trades, notes, settings	belong to the trader; changed only by the trader

5.2 Where each lives

Data	Lives in	Written only by
each tool's results, its screeners and its settings	the tool's own file, with a copy in the shared record	the desk
the market's history	the desk's own file, with a copy in the shared record	the desk
facts about stocks	the shared record	the desk, each kind of fact through its own source
the O'Neil research	the shared record	the research worker
strategies and backtests	qp's file	qp
trades and notes	the journal's file	the journal
keys and passwords	protected files on the box, never copied anywhere	the trader

One writer for each. No two programs write the same file or the same kind of fact, so nothing is overwritten by surprise.

5.3 Every fact carries its time

A fact records what it describes, its value, its source, and **when the desk learned it**. A fact that could not be had says so, with the reason. That is what keeps the record true:

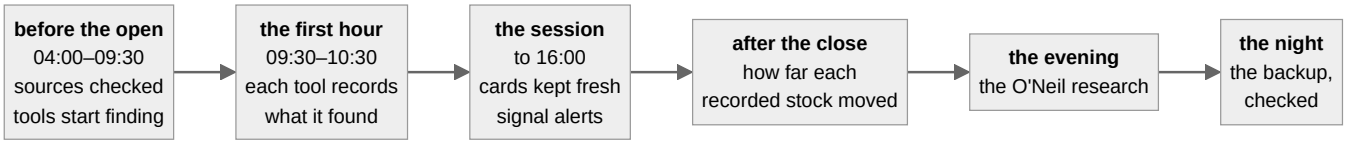
- a tool's morning record holds the facts known at its minute;
- a backtest sees only the facts known before each decision;
- a journal trade sees only the facts known at its entry.

5.4 Time

Every instant is stored in universal time and shown in New York time. One trading calendar, with its holidays and half days, serves every block. Ages and look-backs are counted in trading days, so a weekend never looks like missing data.

---

6. A trading day



When (New York)	What happens	Who
before the first scan	every outside source is checked	the scheduler
04:00 to the close	each tool scans at its pace, cards what it finds, and keeps its cards fresh	the desk
in the first hour	each tool records the stocks it has found, at its own minute	the desk
through the session	saved trading rules are checked against today's stocks, and alerts sent	the scheduler, qp
after the close	how far each recorded stock moved	the desk
the evening	the O'Neil research for every stock the tools have seen	the research worker
when the research ends	a backup of everything, checked	the scheduler
all day	the trader's real trades come in from the broker	the scheduler, the journal

The exact times of every job: the desk's scheduler (design/desk/DESK.md).

6.1 When something goes wrong

Event	What happens
a source fails	the facts it gives show "unavailable, because …", and the cards still build. The health page turns red, and the phone is told
the desk crashes	it restarts in seconds, and every tool's list comes back from its file. A record whose minute passed while it was down is marked missed, never taken late as if on time
the desk does not come back	the check from outside the box tells the phone
the research fails	each step reports on its own. Facts keep their dates, so a card says how old they are
the shared record is down	each tool keeps working from its own file; the copies catch up later
qp or the journal is down	the desk carries on; the phone is told
the server is lost	a new one is set up, and one command restores last night's backup

---

7. Rules that apply everywhere

Each document applies these to its own block. Each was learned from a fault on the live desk.

Rule	Means
honest gaps	a missing value says it is missing, and why; it is never shown as 0, as "no", or as a bad sign
no hindsight	every number uses only what was known at its moment
a label, never a filter	shared facts describe a card; they never change which stocks a tool finds
one writer	every file and every kind of fact has one program that writes it
results stay apart	one tool's results never reach another
one definition	each calculation is written once, in one place
versions	a screener or a model that changes gets a new version, and every record says which version made it
checked on save	a setting that cannot work is refused when saved, with the reason
failures are loud	a job that fails or does not run reaches the phone
one record, at its minute	a tool's daily record is taken only at its minute; a missed one is marked, never faked
no overlap	a job never runs twice at once
trading days, not hours	ages and look-backs are counted on the trading calendar
proven before it ships	every change passes the tests; every move is compared with what it replaces before the old part stops
secrets stay put	keys never appear in a reply, a log, the code or a backup

The faults behind each rule are named in the block documents.

---

8. Running it

The box is one small cloud server with 1 GB of memory and a swap file. Today the desk's programs leave it almost nothing spare; one program for every tool frees most of that memory.

One supervisor keeps every program running, restarts it if it stops, and holds it to its memory budget. **One command** deploys a new version: it backs everything up, restarts the programs in order, checks each one, and stops at the first failure.

Backups go off the box every night and are checked by restoring them. One command puts a night back, and a monthly rehearsal proves it works. Keys are never in a backup: the trader keeps their own copy.

Detail: design/box/BOX.md and design/outside/BACKUPS.md.

---

9. Where the build stands, and the way there

On 2026-10-08, the data half of the plan is built and deployed: the shared record, the research worker, run history and alerts, verified backups, the one address. Most of it runs **in shadow**: beside the old way, compared with it, with nothing depending on it yet. The program half is next.

Block	Today
front door	built; the one address and login work, but the domain and closing the other ports are not done
desk	being built: the way one program holds several tools is built; the tools still run as five separate programs

Block	Today
files on the box	built
qp	mostly built; reading the tools' history from the shared record is built but off
research worker	built, in shadow; the cards still use qp's numbers
journal	mostly built; stops and the snapshot of each trade's entry are not
data sources	partly behind their own code; the rest moves with the desk
shared record	built, in shadow
backups	built
alerts	the phone works for the most important jobs; the health page is not built

The way there. Six steps. Each ships on its own, keeps the desk working, is compared with what it replaces, and can be undone.

Step	What
A	the desk program starts, serving the stopped tools; the old archive program stops
B	the desk takes the running tools, one at a time, each only after a week in which it records exactly what the tool's own program records
C	one scheduler for everything; the old timers are retired
D	the shared facts and the one market read are switched on, each when its comparison is clean
E	the health page, scorecards, live pages, phone-first pages
F	a model to rank each tool's stocks, when the trader chooses one

Detail, with each step's check and undo: design/MIGRATION.md.

---

10. Decisions

Decision	Decided
how the tools run	**one program for every tool** (by the trader, 2026-10-08). The earlier plan, two programs (one for the pages, one for the jobs), would have had to copy everything a page shows between them all day, and would have loaded the code twice. Why in full: design/desk/DESK.md
where a tool's results are kept	in the tool's own file on the box, with a copy in the shared record. A record kept only in a remote database would make every scan and every page depend on the network
the database	Postgres, for the shared record
where the shared record lives	**open**: on Railway for now. It could move onto the box or to AWS; moving is one restore. The trader decides
the server	the 1 GB server; a 2 GB one would remove the squeeze
the address	a free domain name with its own certificate
alerts to the phone	ntfy: faults on one channel, signal alerts on another; a free outside check for when the desk itself stops
the language	JavaScript for the desk and the journal, Python for qp and the research. No rewrite: it adds risk without a trading gain
merging identical scans across tools	dropped: every tool's screeners differ, so there is nothing to merge
one tool's daily record time (T6)	**open**: its record is taken before its screener starts looking. Move the record later, or start the screener earlier: this decides what the tool measures
a model	prepared, not built; the trader may choose one later

---

11. Risks

Risk	What contains it
TradingView's scanner is unofficial and could change or refuse the desk	one piece of code reaches it and refuses a changed answer; it is checked before every day; the desk says plainly when it cannot see
free data has limits (some sources are a day behind, thin, or slow)	each source's code states its limits; the desk falls back where it can and says when it cannot
one program for every tool: a crash stops them all	it restarts in seconds; nothing is lost; a missed record is marked and the phone is told
one server, no spare	nightly backups, checked; a restore in one command, rehearsed monthly; the shared record holds a copy of every record
memory on a 1 GB server	a budget for every program, enforced; the heavy research runs in the evening, when nothing else does
a source that is confidently wrong	sources are cross-checked where they overlap, and every fact keeps its source

---

12. The document tree

Read from the top. Each document names the one above it in its first line, and has its own diagram.

```
ARCHITECTURE.md ....................... the whole desk (this document)
│
├── design/box/BOX.md ................... the box: memory, supervisor, deploy, secrets
├── design/front-door/FRONT-DOOR.md ..... the address, the login, routing
├── design/desk/DESK.md ................. the desk: one program, each tool kept apart
│   ├── design/desk/WEB.md .............. how a page reaches its tool
│   ├── design/desk/HOME.md ............. the landing page, the lists across tools, Manage tools
│   ├── design/desk/TOOL.md ............. one running tool, end to end
│   │   ├── design/desk/tool/SCREENERS.md   its screeners and how it finds stocks
│   │   ├── design/desk/tool/CARD.md ....... the card: every field and its source
│   │   ├── design/desk/tool/REGISTERS.md .. its daily record and the outcomes
│   │   ├── design/desk/tool/LEARNING.md ... the score and the scorecard
│   │   └── design/desk/tool/PAGES.md ...... its pages
│   ├── design/desk/STOPPED-TOOLS.md .... a stopped tool's history
│   ├── design/desk/MARKET.md ........... the market read
│   └── design/desk/JOBS.md ............. the scheduler, job by job
├── design/files/FILES.md ............... every file on the box and its writer
├── design/quant/QUANT.md ............... qp
│   ├── design/quant/CHART.md ........... the page and the print sheets
│   ├── design/quant/LIBRARY.md ......... the checked indicators
│   ├── design/quant/STRATEGIES.md ...... trading rules and how they become trades
│   ├── design/quant/BACKTEST.md ........ the backtest
│   ├── design/quant/SIGNAL-ALERTS.md ... signal alerts
│   └── design/quant/FEEDS.md ........... price bars
├── design/research/RESEARCH-WORKER.md .. the research worker
│   └── design/research/ONEIL.md ........ the O'Neil method, letter by letter
├── design/journal/JOURNAL.md ........... the journal
│   ├── design/journal/IMPORT.md ........ how trades come in
│   └── design/journal/REVIEW.md ........ the page and the analysis
├── design/outside/SOURCES.md ........... every outside source
├── design/outside/SHARED-RECORD.md ..... the shared record
├── design/outside/BACKUPS.md ........... backup and restore
├── design/outside/ALERTS.md ............ alerts and the health page
└── design/MIGRATION.md ................. from today's desk to this one
```

Where to read them. Each document has a PDF in pdf/, in the same tree, and pdf/DESIGN.pdf holds them all in this order. The index, with what is written so far: design/README.md.

Every document has the same parts: its place in the tree; what it is; its diagram; its parts; what it owns and reads; what happens when it fails; what it is built on; and its status.

What the design is built on. The desk that runs today, documented from its code under legacy/: every card field and its source, every calculation, the research method, the O'Neil method and the journal. Each block document names the faults of the old desk that its design prevents. The first design, which this tree replaces, is kept in archive/design-v1/ until the tree carries everything in it.

---

13. Words used here

Word	Meaning
desk	the one program that runs every tool
tool	one way of finding stocks, run as an experiment, with its own history
screener	one question a tool asks TradingView's scanner, such as "stocks up 5 % on heavy volume"
card	everything the desk knows about one stock on one day
benchmark	a plain tool that every other tool must beat
fact	one thing known about a stock or the market, with its source and the time it became known
shared record	the database for what more than one program needs
qp	the chart and backtest program
research worker	the job that does the O'Neil research, then stops
O'Neil method	William O'Neil's way of reading stocks and the market (CANSLIM)
in shadow	built and running beside the old way, compared with it, but nothing depends on it yet

The box

Above: ARCHITECTURE.md §3 (the box around every program) and §8 (running it). **Talks to:** every program on it. The front door is its only open port; the backups carry its files off it.

---

1. What it is

The one server everything runs on, and how it is run. It is a small cloud server with 1 GB of memory. One supervisor keeps every program running inside its own memory budget, one command deploys new code, and the keys live in protected files that never leave it.

The machine	
Machine	AWS EC2 **t3.micro**: 2 vCPU, **912 MiB** of memory
Swap	a **2 GB swap file**, made once by `deploy/add-swap.sh`; the deploy requires it
System	Amazon Linux 2023. Its `python3` is 3.9: the Python programs are tested on 3.9
User and folder	`ec2-user`, `~/Tade-desk-server` (one checkout of the code repository)

What it does:

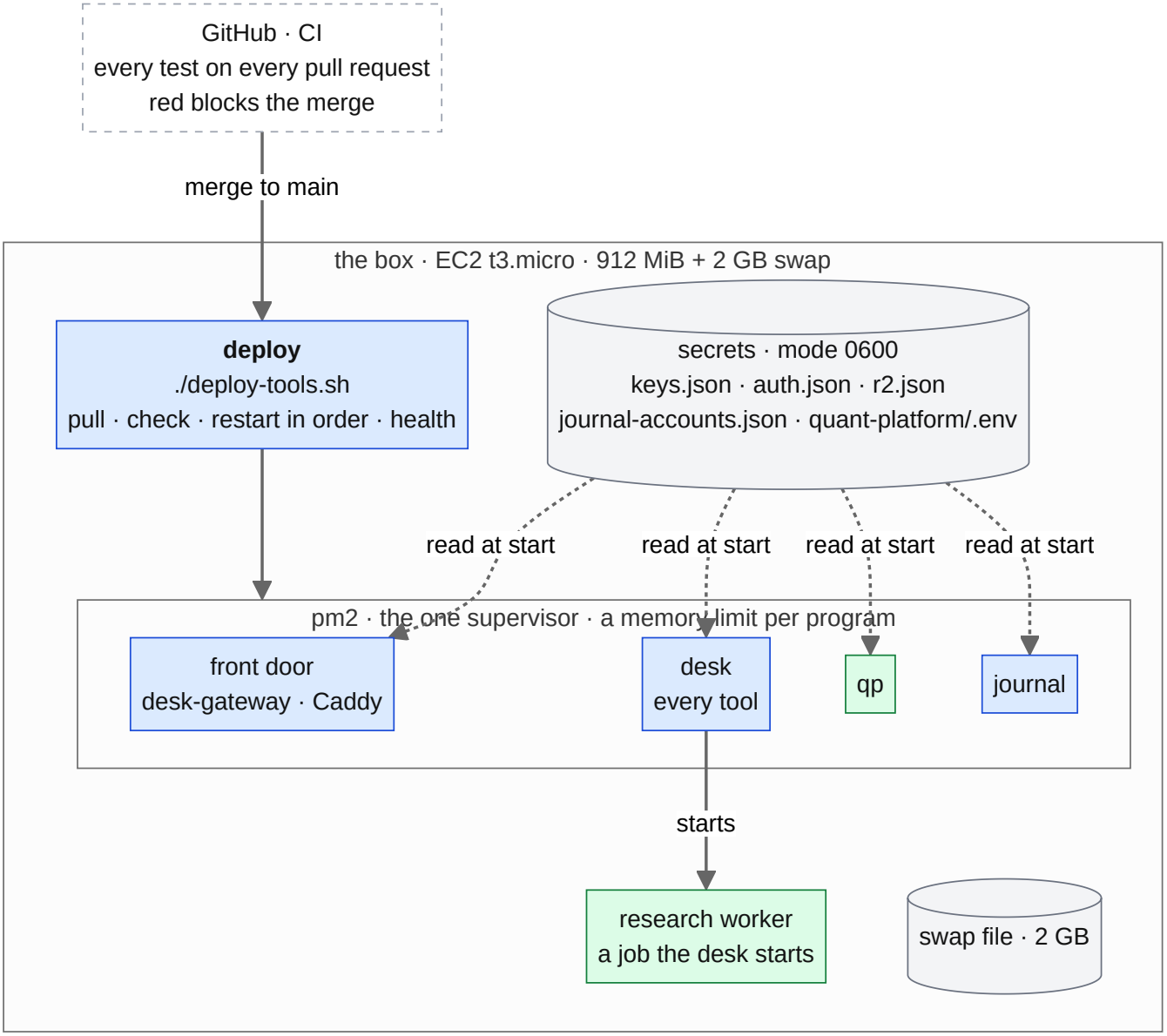
- runs every program of the desk under **one supervisor** (pm2), each with a memory limit;
- deploys the code with **one command**;
- keeps the secrets in files only its owner can read, which never leave the box.

What it never does:

- run a program under two supervisors;
- keep a secret in git, in a log or in a backup;
- accept a deploy in the minutes around a photo (once built, §4.4).

---

2. The diagram



3. The programs and their memory

Measured on 2026-10-06, today's arrangement: about 724 MB of programs, beside the system.

Program	Memory
five tool programs	89–91 MB each
qp	129 MB
the archive	78 MB
the journal	66 MB

Program	pm2 name	Port (local)	Memory budget	pm2 limit
desk (every tool)	`desk`	3000 (home), and one per tool: 3010 … 3120; T1 on 3090 once it moves (WEB.md §3.1)	about 150–250 MB (estimate; measured as tools move in)	to set from the measurement
qp	`qp`	8765	70–150 MB (129 measured)	`deploy/qp.config.js`
journal	`journal`	3100	about 66 MB	160 MB
front door	`desk-gateway`, Caddy	8080; 443 outside	40–80 MB	80 MB (gateway)
research worker	— (a job)	—	100–150 MB for new tickers; 200–400 MB at night	—
system	—	—	about 150 MB	—

About 500–650 MB in all. The night's research run happens at 18:30, when no scan is due, and may use swap. A 2 GB instance removes the squeeze; that is the trader's decision (ARCHITECTURE §10).

Today's limits, until the tools move into the desk:

Program	pm2 limit
T1	280 MB
T2, T6	240 MB
T11	200 MB
T7	180 MB
any other tool	140 MB
the archive	120 MB
the desk's comparison copy	300 MB

pm2 restarts a program that passes its limit. Nothing is started by Manage tools with less than 180 MB available, because on a 912 MiB box the alternative is the kernel choosing what to kill, and it may choose `sshd`.

---

4. Its parts

4.1 The supervisor

pm2 for every program of the desk.

- Every program of the desk is a pm2 process with a memory limit and timestamps in its log.
- `pm2 save` and `pm2 startup` bring them back after a reboot.

Each program has exactly one supervisor, never two. Two supervisors once restarted one program 113,854 times (2026-09-23). The box's own services (sshd; Caddy and the DuckDNS updater, FRONT-DOOR.md) stay under systemd and cron as system services, never also under pm2.

The one exception, to be retired: a systemd timer, `qp-daily.timer`, still runs qp's nightly O'Neil chain. `scripts/retire-old-schedulers.sh` removes it and the pm2 backup cron. The desk's night job then runs the chain and the backup (JOBS.md).

4.2 The deploy

One command on the box: `./deploy-tools.sh`.

It	Why
pulls itself first, and runs itself again if the script changed (guarded against looping)	bash keeps reading the old file, so a change used to take effect one deploy late
reads the tool registry **after** the pull	or it starts the previous deploy's tools
refuses to reset when the branch would take `quant-platform/` back to an older state	a deploy from an older branch once silently reverted qp
installs what each program needs (`npm ci`; `psycopg2-binary` for the research worker when missing)	
backs up every file and the shared record before it restarts anything (to build, BACKUPS.md)	a restart is when migrations run, so there is always a copy from just before a schema changed
restarts in order: the desk (today: each tool through `deploy/tool-ctl.sh`), the comparison copy when one is listed, the archive (until it retires), the journal, the front door, qp (through `deploy/qp-restart.sh`, only if qp is running older code)	
copies the Alpaca pair into `quant-platform/.env` (`scripts/sync-qp-env.js`)	so qp and the desk never hold different keys
checks every program's `/health`, and what each data feed actually returned (`/api/feedcheck`, fetched, not inventoried)	a key that is present is not proof that it works

To build. The deploy refuses to run within 15 minutes of any tool's capture minute on a trading day, unless forced. A deploy at a photo minute costs that tool a day of research.

CI, before the deploy. Every pull request runs every test: Node (Jest) and Python (qp's `run_all`, the research worker), against a real Postgres. Red blocks the merge.

4.3 The secrets

File	Holds	Set with
`data/keys.json`	the shared keys and settings of the desk: `databaseUrl`, `alertNtfyTopic`, the area switches (`factsMode`, `marketMode`, …), `workerShadow`	`node scripts/set-setting.js <name>`: asks with echo off, checks a database address before saving
`data/auth.json`	the password hash and the cookie's signing secret	`scripts/set-password.sh`
`data/r2.json`	the backup bucket's keys	`scripts/set-r2-keys.sh`
`data/journal/journal-accounts.json`	the journal's Alpaca accounts	the journal's Accounts page
`quant-platform/.env`	qp's feed keys	copied by the deploy from the desk's keys

The rules:

- **Mode 0600**, gitignored, never printed, never logged, never backed up.
- The trader keeps a copy off the box; a restore puts the secrets back from that copy (BACKUPS.md).
- Secrets are entered on the box, never pasted into a chat or a log.

To build. One secret store, written by one command (`desk secret set <name>`) and read by every program, so that no key is copied between files.

4.4 Ports

Inside the box	Outside
each program keeps its own local port; programs call each other on `127.0.0.1`	**today:** the programs' ports are still open to the internet. **The design:** only 443 (the front door) is open

The login treats a call from the box itself as local only when it did not pass through the front door (FRONT-DOOR.md).

4.5 Logs

pm2 keeps each program's output, with timestamps. A pm2 restart count is the only trace a memory-limit kill leaves, so the deploy prints each program's restart count.

To build:

- rotated logs;
- the last errors of every program, shown on the health page (ALERTS.md).

4.6 The old helper scripts, and where each goes

Script today	Where it goes
`scripts/set-password.sh`, `set-r2-keys.sh`, `set-alpaca-keys.js`, `set-setting.js`, `sync-qp-env.js`	one secret store and `desk secret set` (§4.3)
`deploy-tools.sh`, `deploy/qp-restart.sh`, `deploy/add-swap.sh`	kept: the one deploy; swap is made once
`deploy/tool-ctl.sh`	retired as the tools move into the desk: a tool is a context, not a process. Manage tools adds it to the desk instead (HOME.md)
`show-screeners`, `check-screeners`, `why-empty`, `try-filter`, `field-values`, `verify-tv-fields`, `probe-*`	the screener builder's Test, the adapters' live column list, and the scheduled health checks (SCREENERS.md, SOURCES.md)
`audit-floor`, `purge-sub-floor`, `backfill_signals`, `rebuild_training`, `convert_legacy_to_r4.py`, `split-tool-history`, `rebuild-tool-db`	one-time repairs, kept as scripts until the import of the old data is complete (MIGRATION.md)
`analyse-bias`, `analyse-news-chain`, `card_lookup.py`, `verify_irdm_score.py`, `export-training.js`, `archive-values.js`	the register exports and the API: a card as of a day, the R1 register, the score's breakdown
`scripts/desk.js` (restore), `backup-r2.js`, `restore-r2.js`, `import-registers.js`, `facts-report.js`, `worker-report.js`	kept (BACKUPS.md, SHARED-RECORD.md, MIGRATION.md)
qp's `run_daily.py`, `run_backfill.py`, `run_edgar.py`, `run_13f.py`, `run_sic.py`, `run_status.py`, `run_fields.py`	the research worker's steps, started by the desk's scheduler; their checks on the health page (RESEARCH-WORKER.md)
qp's `tools/` (compare server, catalogue, strategy checks, recovery, rank sweep), `exec-compare.js`, `fill-compare.js`, `check-feeds.js`	kept with qp as they are (QUANT.md)

---

5. Contracts

It provides	To
a supervised process with a memory limit, restarted after a crash or a reboot	every program

It provides	To
one deploy command with health checks	the trader
the secret files, mode 0600	every program (read at start)
`127.0.0.1` between programs	every program
the 2 GB swap file	the night's research run

6. When it fails

Event	What happens
a program passes its memory limit	pm2 restarts it. The desk brings every tool's list back from its file; the restart count shows it
memory runs short	swap absorbs a peak. Manage tools refuses to add a tool with less than 180 MB available
a deploy fails midway	it stops at the failing step and says which. The programs already restarted keep running; the rest keep their previous code
the box reboots	pm2 brings every program back; jobs whose slot passed are recorded as missed and alert (JOBS.md)
the box is lost	a new box: install, put back the secrets from the trader's copy, then `node scripts/desk.js restore <date>` (BACKUPS.md). The shared record still holds the copy of every photo taken since the last backup

7. Built on

The old desk:

- today's architecture;
- the deploy script's own recorded lessons (§4.2);
- the code repository's `deploy/README.md`;
- the qp operations;
- the journal operations.

What failed there:

- **Two supervisors:** 113,854 restarts on 2026-09-23.
- **A qp restart took 3 min 20 s:** the port check refused the port while old connections lingered. Fixed on 2026-10-06.
- **Keys pushed to GitHub** by an old backup (S-2).
- **Tools stopped for lack of memory:** five tool programs at about 90 MB each.
- **No CI** until step 1.

8. Status

Part	Status
the machine, swap, pm2, the deploy	**built**
CI	**built** (step 1)
the night handed to the desk's scheduler	**built, not switched**: `scripts/retire-old-schedulers.sh` is ready; the old timer still runs
the photo-window guard on the deploy	**to build** (migration step A)
one secret store	**to build**
only 443 open	**to do by the trader** once the domain is set (FRONT-DOOR.md)
the desk as one pm2 process for every tool	**being built** (DESK.md, MIGRATION.md steps A and B)

The front door

Above: ARCHITECTURE.md §4.1. **Talks to:**

- the trader's phone and laptop;
- the desk, qp and the journal, behind it;
- the box it runs on.

---

1. What it is

The one address the trader uses, on a phone or a laptop:

- **HTTPS** on a free domain name;
- **one login** for every program;
- **every program behind one port, by path**.

What it owns:

- the domain and its certificate;
- the routing rules;
- the login's password file.

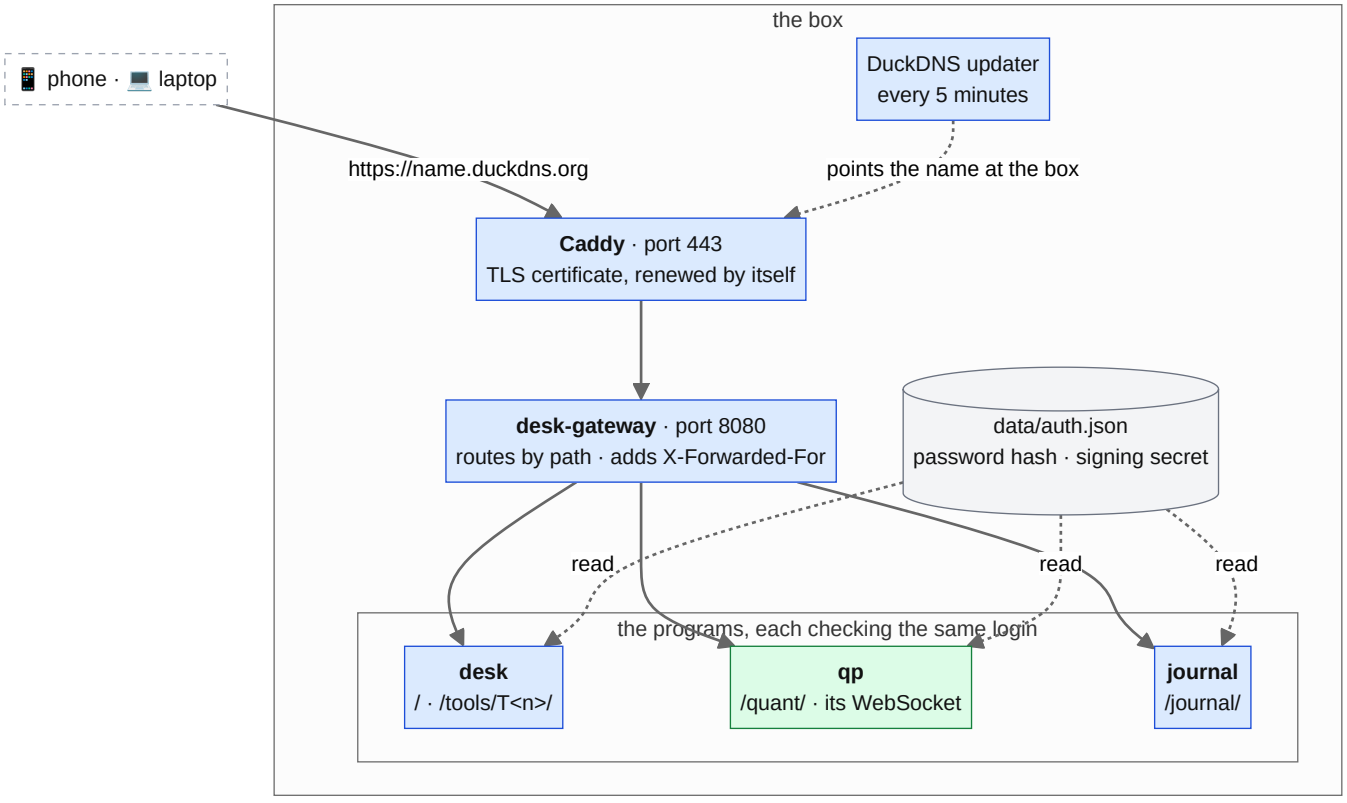
What it never does:

- hold state;
- compute anything;
- let a request in without the login.

If it stops, the desk keeps scanning and photographing, and only the pages are out of reach until it restarts.

---

2. The diagram



---

3. Its parts

3.1 The address and TLS

- **The name.** A free DuckDNS name, `<name>.duckdns.org`. A small updater (cron, every 5 minutes) points the name at the box's address. Its token is kept in `~/.duckdns/token`, mode 0600.
- **TLS.** Caddy serves HTTPS on 443 with a certificate it obtains and renews itself, and passes every request to the gateway on 8080. Caddy passes the Host through, so the pages know which address the browser used.
- **Ports.** Once the name answers, only 80 and 443 stay open in the AWS security group. The program ports (3000–3120, 8765, 8080) close to the internet.

Caddy and the DuckDNS updater are system services: systemd and cron. They are the box's own services, like `sshd`, not desk programs. Each has one supervisor, as every program does (BOX.md §4.1).

3.2 Routing by path

The gateway (`src/shell/gateway.js`) holds no state.

Path	Goes to
`/`, `/login`, `/logout`, `/screeners…`, `/market`, `/status`, `/settings`	the desk's home (today `/`, `/login` and `/logout` only; the rest **to build** with home in the desk, WEB.md §3.2)
`/tools/T<n>/…`	that tool, in the desk (running or stopped)
`/quant/…`	qp, including its live chart WebSocket (`/ws/live`)
`/journal/…`	the journal

How pages that write absolute paths keep working. Pages ask for `/api/…`, `/desk.js` or `/static/…` at the root of their own address.

- **Sub-requests.** The browser sends the page's address as `Referer`, and the gateway sends the request to that page's program. Only a Referer from the same address is followed.
- **Navigations.** A link or `window.open` to an absolute path, from a page inside a program, is redirected under that program's prefix. `/`, `/login` and `/logout` always belong to the desk.
- **Redirects.** When a program redirects to one of its own paths (`/login?next=…`), the gateway keeps it under that program's prefix.
- **Links between programs.** Every HTML answer sets the cookie `desk_shell` to the address the browser used. The page shell then links by path (`/tools/T2/`, `/journal/`, `/quant/`) instead of by port (WEB.md §3.3).

3.3 The one login

One password, one cookie, checked by every program:

- the desk (`src/auth/index.js`);
- the journal (the same module);
- qp (`quant-platform/chart/auth.py`). It is pure ASGI, so its WebSocket is covered too.

Part	Rule
`data/auth.json`	mode 0600, gitignored, written by `scripts/set-password.sh`. It holds the password as a **scrypt** hash (N 16384, r 8, p 1) and a random signing secret. It is re-read when it changes, so a new password applies everywhere at once
the cookie `td_auth`	`<expiry>.<HMAC-SHA256(secret, "td1.<expiry>")>`, valid 30 days. A cookie, not Basic auth, because cookies ignore the port: signed in on one program is signed in on all. `HttpOnly; SameSite=Lax`, and `Secure` once the desk is served over HTTPS
no password set	**locked, not open**: every outside request gets a page saying how to set one
a new password	writes a new secret, which signs everyone out
minimum length	10 characters
let through without the cookie	`/login`, `/logout`; calls from the box itself (`127.0.0.1`, `::1`) that did not pass through the front door
never a bypass	every request the gateway forwards carries `X-Forwarded-For`, which every program reads as "not local". A request through the front door is therefore never treated as the box calling itself
failed attempts	counted per address and slowed down

To build:

- a request that changes anything must carry the desk's own `Origin`, so that no cross-site form or script can act as the trader;
- a list of sessions, and "sign out everywhere".

3.4 The page shell, across programs

Every page of every program carries the same top bar (`public/desk.js`, `public/desk.css`). It names the programs and links to them by path when the page came through the front door, and by port when it did not. The bar and its assets are never cached without a check, so a deploy

shows at once. Detail: WEB.md §3.3.

---

4. Contracts

It owns	Form
the routing table	built from the tool registry and the apps list (FILES.md)
the login	`data/auth.json`; the `td_auth` cookie
the address	the DuckDNS name; Caddy's certificate

Route	Does
`GET /login`, `POST /login`, `POST /logout`	sign in, sign out (served by the desk)
everything else	forwarded by path (§3.2)

---

5. When it fails

Event	What happens
the gateway stops	pm2 restarts it in seconds. The programs keep running; pages are out of reach meanwhile
Caddy stops, or its certificate cannot renew	systemd restarts Caddy. A certificate that cannot renew is reported on the health page before it expires (to build)
the box's address changes	the DuckDNS updater points the name at the new address within 5 minutes
a program behind it is down	the gateway answers "T2 is not answering (port 3010)", naming the program and its port
no password is set	every program is locked until one is set

---

6. Built on

The old desk. Every program listened on a public port of its own, over plain HTTP. There were thirteen: the five running tools, the archive on the six stopped tools' ports, qp on 8765 and the journal on 3100.

What failed there.

- **No login at all (S-1).** Until the 2026-10 review, anyone who found a port could change settings, keys and strategies. qp's log already showed internet scanners on 8765. The one login above fixed it (S-1).
- **Cross-program pages.** They needed credentialed CORS and a `fetch` wrapper to carry the cookie across ports.

Documents:

- the code repository's `docs/SHELL.md`;
- `src/shell/gateway.js` and `tests/shell.gateway.test.js`;
- `scripts/setup-domain.sh`;
- `src/auth/index.js`;
- `quant-platform/chart/auth.py`.

---

7. Status

Part	Status
one login in every program, locked when unset	**built** (S-1)
the gateway: one port, every program by path, never a login bypass	**built and running** (`desk-gateway`, step 14)
the domain and Caddy	**script written, not run.** It installs Caddy with `apt-get`, but the box runs Amazon Linux 2023, which uses `dnf`: **fix this before running it** (install Caddy's static binary or its `dnf` package)
closing the program ports	**the trader's to do**, after the domain answers
the `Secure` cookie flag	follows HTTPS
the `Origin` check; the session list and "sign out everywhere"	**to build**

The desk

Above: ARCHITECTURE.md §4.2 (the desk), §1 (how the desk learns) and §6 (a trading day).

Talks to:

- the front door, which brings it every page and API call;
- the files on the box: it is the only writer of each tool's file;
- the data sources and the shared record;
- the research worker, qp and the journal, whose timed work it starts;
- the backups and the alerts.

---

1. What it is

One program that runs every tool. For each running tool it does what a tool's own program does today:

- serves its pages and its API;
- runs its screeners on schedule;
- builds and refreshes its cards;
- keeps its live list and its shortlist;
- records what the tool found each morning (its photo) and how those stocks moved (§4.5);
- scores its cards.

For each stopped tool it serves the history, read-only. It is also the desk's only scheduler.

What it owns:

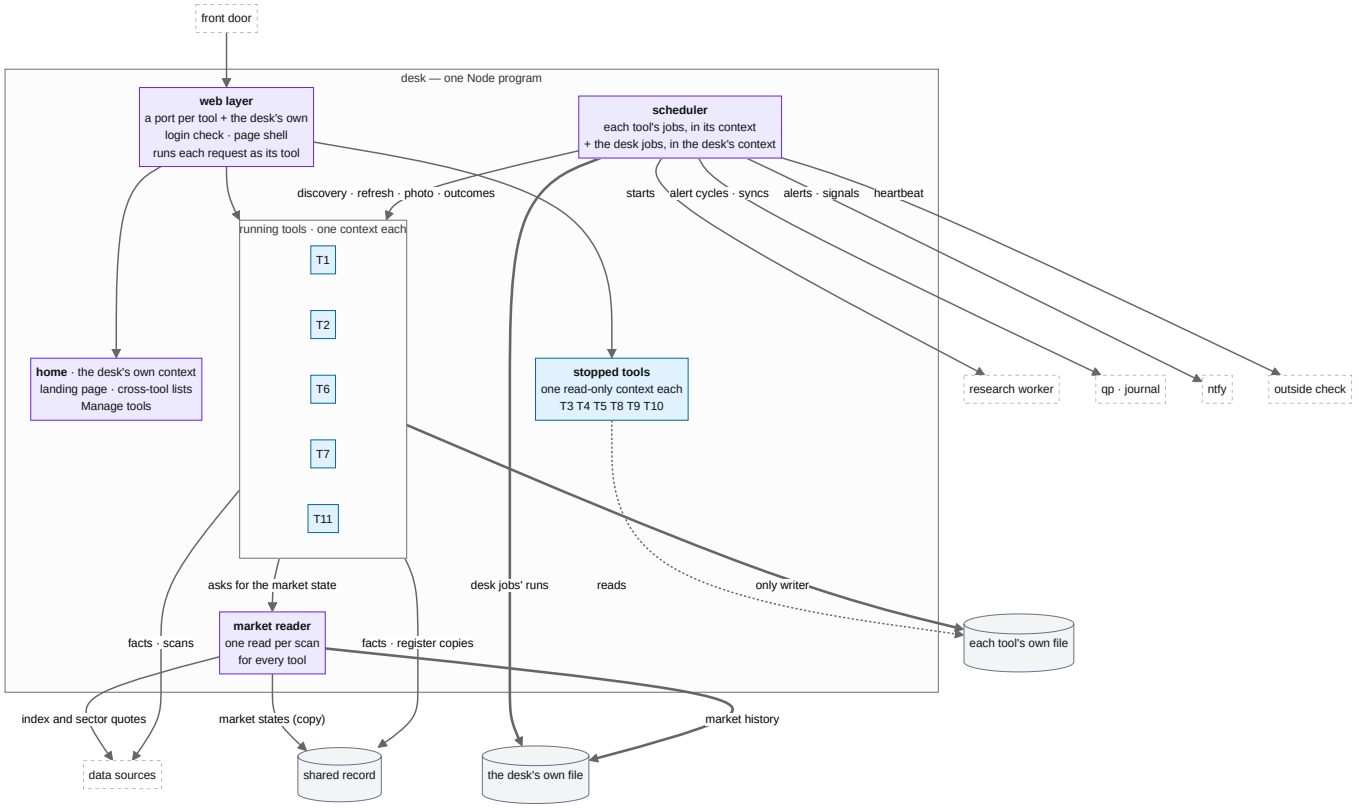
- each tool's own file, of which it is the only writer;
- its own file, for the desk jobs' runs and alerts, the market history and the desk's settings;
- the live state of every tool, in memory and saved to each tool's file after every change.

What it never does:

- **compute O'Neil data:** the research worker does;
- **evaluate a trading strategy:** qp does;
- **place an order;**
- **let one tool see another's results.**

---

2. The diagram



3. The tool context: how one program holds many tools

The desk keeps tools apart with **contexts**. A context is everything that used to be one tool's program-wide state:

A context holds	Today, in a tool's own program, this was
the tool's identity: id, name, port, capture times, scorer	the environment the program was started with (`TOOL_ID`, `PORT`, …)
its file: the open database handle	the one database opened at start-up
its live list (R0)	a map in the program's memory
its scan status, market snapshot, hot sectors, scorer state	module variables
its jobs	the program's own scheduler
its role: running, stopped (read-only), or a comparison copy	—

How code finds its tool.

- Every piece of work runs inside one context: a page request, a job, a scan.
- Every request runs as its tool, and every job as the tool that registered it.
- Code asks "which tool am I working for?" and gets the context's answer, across every `await` and every timer.

Why two tools cannot mix. A database query made outside every context is an error in the desk, not a quiet write into T1's file. So a tool's results can only ever reach its own file.

The desk's own context. Home and the desk jobs run as the desk itself, with its own file. That is why every tool, T1 included, can be stopped.

The comparison copy. A context can also be a **copy** of a tool that still runs as its own program. A copy:

- follows the tool's screeners, settings and stars, read-only from the tool's file;
- writes only its own file;
- sends no alert and writes nothing shared;
- is compared with the tool's own photo every day.

This is how each tool proves it takes the same photo in the desk before it moves (MIGRATION.md step B).

Built: `src/toolContext.js` (migration step 15a). The copy and its daily comparison: `src/worker/` (step 15b).

4. Its parts

Part	Job	Detail
web layer	brings every page and API call to its tool: one listener per tool port (so today's addresses keep working) and the desk's own; the login check; the page shell; live updates to open pages	WEB.md
home	the landing page; the lists across all tools (the unified shortlist, the screener report, the comparison); Manage tools (add, start, stop, pause, edit, delete, restore)	HOME.md
running tools	one tool end to end: its screeners and discovery, the card, the live list, the shortlist, the photo, the outcomes, the score, its pages and API	TOOL.md
stopped tools	a stopped tool's stored registers and history, read-only, on its own address	STOPPED-TOOLS.md
market reader	the indexes, the regime, the short-, mid- and long-term reads, the 15 sectors and hot sectors: read once per scan for every tool, stamped on every card, kept with its history in the desk's own file (R2 is that history at fixed minutes)	MARKET.md
scheduler and desk jobs	the only scheduler: every tool's jobs in its context, and the desk jobs (research runs, the night, the restore test, schedule changes, journal sync, qp's alert cycles with their signals pushed to the phone, the health checks, the heartbeat); every run recorded; failed, missed and hung runs alerted (ALERTS.md)	JOBS.md

4.1 Start-up

When the desk starts, it does the following in order. Every step is per tool, and none of them waits on the network.

1. **Read the tool registry.**
   - For each **running tool**:
     1. open its file;
     2. bring its schema up to date;
     3. seed its screeners if it has none (only for the built-in tools);
     4. bring back today's live list from the file;
     5. register its jobs.
   - For each **stopped tool**: open its file **read-only**.
   - **During the move** (MIGRATION.md steps A and B), the running tools are those listed in `deskTools`, and each tool listed in `workerShadow` opens as a copy.
2. **Open the desk's own context.** Register the desk jobs.
3. **Record missed jobs.** A job whose slot passed while the desk was down is recorded as missed and alerts. A photo is never taken late as if on time.
4. **Scan at once, if the session is open.** If it is between 04:00 and the close on a trading day, each running tool scans now, so a restart does not leave a list empty until the next scheduled scan.
5. **Listen on every tool's port** and the desk's own.

4.2 Containment: one tool cannot take another down

- **Every job of every tool runs wrapped:**
  - an error is recorded in that tool's run history, with its stack;
  - it alerts if the job is one that alerts;
  - it never ends the program.
- **Errors outside every job** are logged with their stack, and the program goes on.
- **Heavy work stays out of page requests.** Exports and long checks run as jobs or in pieces, so one page cannot hold up every tool's scans.
- **Bad configuration is refused on save** (HOME.md, SCREENERS.md), so a broken screener cannot reach the scans.

4.3 Memory

Budget. One copy of the code serves every tool: about 150–250 MB for the desk with five running tools (an estimate, to be measured as tools move in). Today the same work takes five programs at about 90 MB each, plus the archive.

What each tool costs is its live list and its caches: a few megabytes.

What is shared in memory across tools:

- the facts that do not depend on a tool (short interest, borrow, the O'Neil readings), cached once;
- the market read, made once per scan.

4.4 What changes for each tool's code

Almost nothing: the screener, card, register and page code is the code that runs today. What changed is where each tool's state is kept (§3), so the same code runs once per tool, in that tool's context.

4.5 The research each tool runs

Every running tool runs the same method, kept exactly as the old desk learned it:

- **The record of the day** (the photo, R1). At the tool's own minute in the first hour (today between 09:36 and 10:16), every card on its list is frozen exactly as it is known then. Only these records judge the tool.
- **Two entry minutes,** A and B, a few minutes after the record.
- **The outcome** (R3). After the close, the desk measures how far each recorded stock moved, up and down, from each entry minute to the close, in units of the stock's average daily range (ATR). A move of 1.3 ATR or more is a good move. There is no stop, target or exit in this measure: it judges what a tool finds, not a trade.
- **A benchmark tool** that every other tool must beat by a clear margin, and a **mirror** for each screener (the same setup facing the other way), to test whether its direction is real.
- **The order of work:** collect for about a month; cut the screeners whose stocks do not move; only then work on entries, stops and targets.

The exact measures: TOOL.md, and below it REGISTERS.md and LEARNING.md.

4.6 A trading day, by the clock

New York time, on trading days. Each job and its settings: JOBS.md.

When	Job	Who
03:45	every outside source checked	desk jobs
04:00–09:00 every 30 min; 09:00–10:00 every 5; 10:00–16:00 every 15; 16:00	discovery: each screener only inside its own window	each running tool, with one market read
04:00–16:00 every 5 min	every card re-quoted and re-scored	each running tool
04:00–16:00 every 5 min, while any wait	research for the stocks carded for the first time	desk jobs → research worker
09:00–15:30 every 30 min	the market model, provisional	desk jobs → research worker
09:25–16:05 every minute, while alerts are on	qp's alert cycle	desk jobs → qp
one minute before each tool's record	the shortlist rule	each running tool
each tool's minute (09:36 … 10:16 today)	**the record of the day** (R1)	each running tool
09:26, 09:31 … 10:01	R2: the market at fixed minutes, a view of the market history	the market reader
16:05	**the outcomes** (R3)	each running tool
16:50	capture-time changes made during the session take effect	desk jobs
18:30	the night: the O'Neil research, then the backup	desk jobs → research worker, backup
every minute, each account on its own interval	the journal's sync	desk jobs → journal
every 5 min	the heartbeat; the box's memory and disk	desk jobs
the 1st of each month, 09:10	the restore rehearsal	desk jobs
00:00	each tool's list starts a new day	each running tool

---

5. Contracts

It owns	Form
each tool's own file	SQLite, one per tool; the only writer (FILES.md)
the desk's own file	SQLite: the desk jobs, their runs, the desk's alert state; the market history; the desk's settings (FILES.md §3.2)
the live state of every tool	memory, saved to each tool's file after every change
the routes of every tool and of home	PAGES.md, HOME.md, WEB.md

It reads	From
the tool registry	`tools.config.json` and `data/tools.local.json` (FILES.md)
facts, O'Neil facts, market states	the shared record (SHARED-RECORD.md)
candidates, quotes, news, short interest, borrow, bars	the data sources, through their adapters (SOURCES.md)
the secrets	the secret files (BOX.md §4.3)

It starts	When
the research worker	the research jobs (JOBS.md)
qp's alert cycle	every minute in the session while alerts are on
the journal's sync	on each account's schedule

It starts	When
the backup	after the night's research run

6. When it fails

Event	What happens
the desk crashes	pm2 restarts it in seconds. Each tool's list comes back from its file; every run is in its file. A slot that passed while it was down is recorded as missed and alerts; a missed photo is never taken late
a tool's job throws	recorded in that tool's run history with its stack; alerts if it is a job that alerts; the other tools are unaffected
it passes its memory limit	pm2 restarts it, as a crash
the shared record is down	each tool works from its own file and fetches its facts itself; the copies wait; the health page says so
a data source fails	that source's facts show `unavailable` with the reason; the cards still build (SOURCES.md)
the front door is down	the desk keeps scanning and photographing; only the pages are out of reach
a scan outlasts its slot	the next slot is skipped and recorded; two scans of one tool never run at once
TradingView changes or drops a column	its adapter refuses the answer, and the scan report says why; no card gets shifted values (SOURCES.md)

7. Built on

The old desk. Every tool was its own program: five running tools at about 90 MB each, plus the archive serving the six stopped tools. Each program had:

- its own scheduler;
- its own copy of the pipeline;
- its own market read;
- its own caches.

The detail:

- today's architecture;
- TOOL-LIFECYCLE;
- MONITOR.

Why one program, not two

What the earlier design wanted, and why. It split the desk into two programs, desk-web for the pages and desk-worker for the jobs, for three reasons:

1. **Memory:** six programs on a 1 GB box.
2. **Speed and safety:** a slow scan must never freeze the pages, and the scheduler must never run twice.
3. **Tools as rows:** adding or stopping a tool should not mean a program, and a restart should lose nothing.

What building it showed.

- **The split was the hardest part.** Two programs share no memory. Everything a page shows from the scans (the live lists, the scan status, the market read, the jobs) would have had to be copied into the database all day, and every button (scan now, a star, a bias, a job switched off) would have become a message to the other program. That copying was the largest and riskiest part of the plan.
- **It broke one writer per file.** Both programs would have written each tool's file.
- **The speed reason hardly applies here.** A scan mostly waits on the network, not on the processor; each tool already serves its pages while it scans.
- **Two programs save less memory than one.** The saving comes from loading the code once; two programs load it twice.

What one program does instead. It keeps every one of those reasons: one copy of the code for every tool; one scheduler, which can never run twice; a tool as a context, not a process; and each tool's list saved to its file after every change, so a restart loses nothing. It also drops the copying, and keeps one writer per file.

Its risks. A crash stops every tool for a few seconds: the supervisor restarts the desk, and every list comes back from its file; a record whose minute passed is marked missed, and the phone is told. A heavy page could slow a scan: heavy work is kept out of page requests (§4.2). If that ever shows, the same code can still be split into two programs later, because the contexts make it possible.

Decided by the trader on 2026-10-08 (ARCHITECTURE §10).

What failed there

- **Memory.** Tools were stopped for lack of memory: each was a whole copy of the program.
- **The live list was lost on restart.** It lived only in memory, until a checkpoint and a scan at start-up were added.
- **T1 could not be stopped.** It carried the landing page, Manage tools and every desk-wide job.
- **A capture-time change needed a restart.** It waited until 16:50, because a restart in the session emptied the list.
- **The archive.** It served stopped tools by loading a separate copy of the modules for each one ("require-cache surgery").
- **The market was read five times a scan.** Five tools read the same ETFs minutes apart, and the hot sectors were forgotten on every restart.

---

8. Status

Part	Status
the tool context	**built** (step 15a): one program can hold several tools, each kept apart; tested with two tools in one program
the comparison copy and its daily photo check	**built** (step 15b), in a separate program (`desk-worker`) that starts when a tool is listed in `workerShadow`. As a context of the desk: **to build** (migration step B)
stopped tools served by the desk	**to build** (migration step A): the desk's first job, in place of the archive
the desk running tools for real: pages, API and scans	**to build** (step B): tool by tool, after a clean week of photos
home and the desk jobs in the desk's own context	**to build** (step B, with T1, last)
the market reader	**in shadow** (step 6): one read stored per scan; each tool still reads its own (MARKET.md)
the scheduler	**partly**: run history and alerts built; one scheduler per tool until step B (JOBS.md)

Files on the box

Above: ARCHITECTURE.md §4.3 (files on the box) and §5.2 (where each kind of data lives).

Talks to:

- **its writers:** the desk, qp and the journal, each writing only its own files;
- **readers:** the research worker, which reads the tools' files read-only;
- the backups, which copy every file every night.

---

1. What it is

Every file the desk keeps on the box: what each one holds, and the **one program that writes it**.

Each tool's own file is the record of that tool's results.

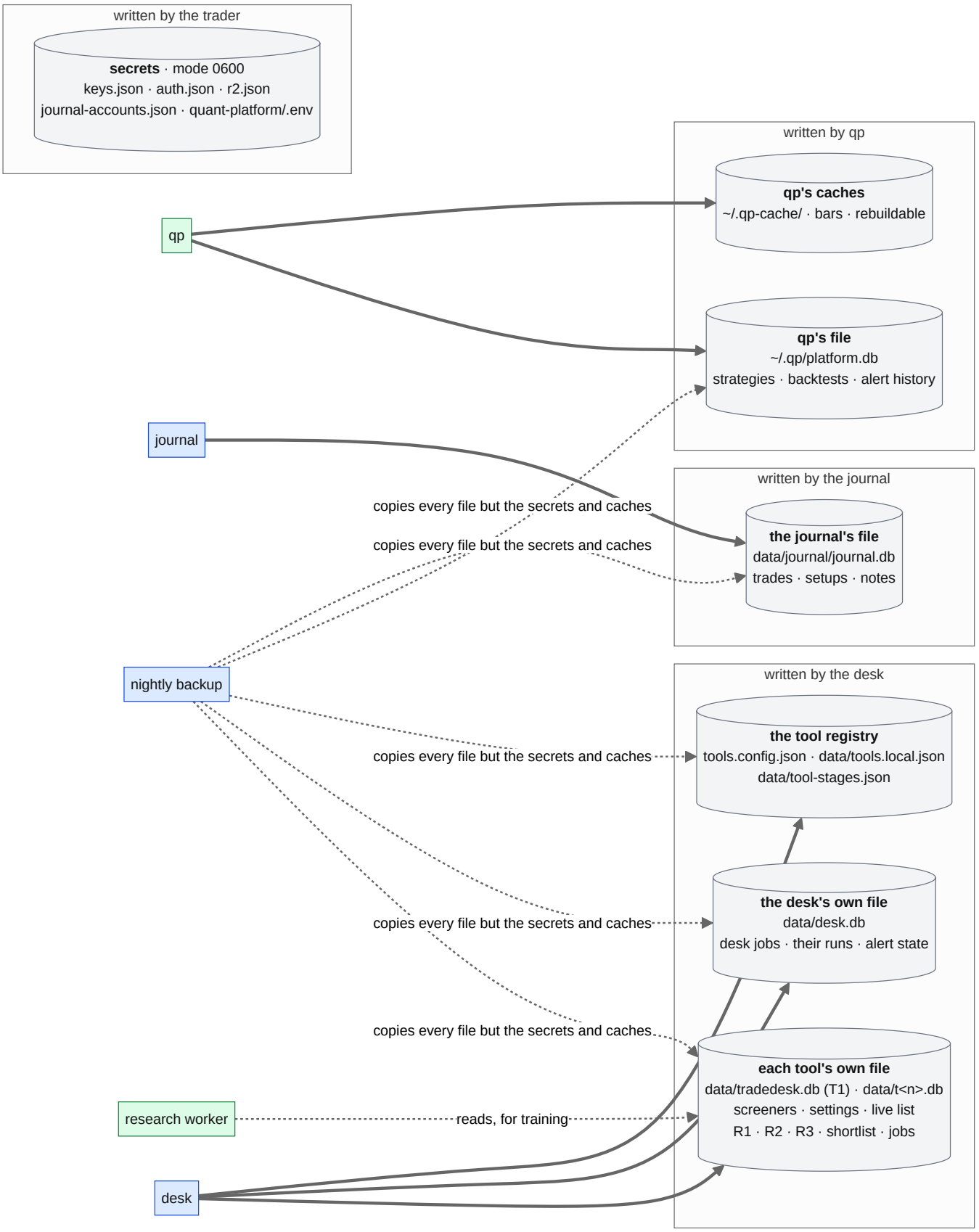
- It works without the network.
- It keeps each tool's results apart by construction, because no tool can open another's file.
- A copy of its registers goes to the shared record at the moment it is written, and the copy is compared with it every day.

The rules for every file:

- **one writer;**
- each change is saved whole, or not at all;
- reading never blocks writing (SQLite's WAL mode);
- its layout changes only through numbered steps, each recorded in the file;
- no secret is ever kept in a data file.

---

2. The diagram



3. The files

3.1 Each tool's own file

T1's file is `data/tradedesk.db`. Every other tool's is `data/t<n>.db`.

- It is SQLite, written only by the desk, in that tool's context.
- A stopped tool's file is opened read-only.

Table	Holds	Detail
`screeners`	the tool's screeners: key, name, rules, sort, size, run and check windows, mirror, list-only, paused	SCREENERS.md
`screener_versions`	every saved version of every screener, so each photo can be read from this file alone (**to build**; today the versions are kept only in the shared record)	SCREENERS.md
`settings`	the tool's settings: its floor override (today: its own floor), shortlist auto-rule, its name and pause; each change a new version (**versions: to build**). Until the one market read is switched on, also its own hot-sector and sector-bias thresholds, which then become the desk's	PAGES.md §Settings
`r0_checkpoint`	the live list of the day, saved after every change (today: after every full scan), so a restart loses nothing	REGISTERS.md
`r1_frozen`	R1: every card of the day, frozen at the capture minute, one row per ticker	REGISTERS.md
`r2_market_snapshots`	R2 as each tool wrote it until the one market read is switched on; kept as history. After that, R2 is read from the market history in the desk's own file	REGISTERS.md
`r3a`, `r3b`	R3: the outcomes from entry A and from entry B	REGISTERS.md
`shortlist`	the day's picks, starred or chosen by the auto-rule	REGISTERS.md
`r4a_train`, `r4b_train`	training rows uploaded by hand (`source = upload`). R4 itself is R1 joined to R3, computed on request	LEARNING.md
`scheduler_jobs`	the tool's jobs: schedule, on/off, the default it was saved with	JOBS.md
`job_runs`	every run of every job: start, end, result, error with stack (kept 30 days)	JOBS.md
`alert_state`	which causes are alerting now, so an alert is sent once	ALERTS.md
`migrations`	one-time repairs that have run	—
`analysis_model`	dead: left from an older app; not read, dropped by a migration	—

Copied to the shared record at the moment they are written, and compared every day:

- `r1_frozen`;
- `r3a` and `r3b`;
- `shortlist`;
- the screeners and their versions.

3.2 The desk's own file

`data/desk.db` (**to build**). It holds what belongs to the desk rather than to any tool, written only by the desk in its own context:

Table	Holds	Detail
`scheduler_jobs`, `job_runs`, `alert_state`	the desk jobs, their runs, the desk's alert state	JOBS.md, ALERTS.md
`market_states`	the market history: one state per market read, with its instant	MARKET.md
`hot_sector_days`	each sector's hot state per trading day	MARKET.md
`settings`, `setting_versions`	the desk's settings (the tradability floor, the market thresholds), each change a new version	TOOL.md §3.1, MARKET.md §3.14

Copied to the shared record at the moment they are written, and compared every day: `market_states` and `hot_sector_days`.

Today the desk jobs run in T1's program and are recorded in T1's file, and the market history is kept only in the shared record. That is one of the reasons T1 cannot be stopped.

3.3 The tool registry

File	Holds	Written by
`tools.config.json` (in the code repository)	the built-in tools: id, name, port, flags (`enabled`, `archive`, `scorer`), accent, description, capture times and why	a code change
`data/tools.local.json` (on the box)	tools added from the page, and overrides of any tool: name, description, accent, capture times, enabled, deleted, pending restart	Manage tools (HOME.md)
`data/tool-stages.json`	each tool's stage override (validated, watch, retired …)	the landing page's stage control

Everything that lists tools reads the **merged** registry (`src/toolsRegistry.js`), so the pages, the deploy, qp and the desk cannot disagree. A tool added from the page gets:

- an id one past the highest ever used, so an id and its history are never reused;
- the next free port.

3.4 qp's files

File	Holds	Detail
`~/.qp/platform.db`	`strategies`, `backtests`, `backtest_trades`; `alert_seen` and `alert_history` (since step 12, so a restart never repeats an alert)	QUANT.md
`~/.qp/strategies/*.json`	strategies exported as files	STRATEGIES.md
`~/.qp-cache/`	bars and the O'Neil caches: **rebuildable**, not backed up	FEEDS.md
qp's operations log	what qp did; rebuildable	QUANT.md

3.5 The journal's file

`data/journal/journal.db`. It holds:

- the trades (`journal_trades`), setups, day notes and fee profiles;
- the leftover tables of an older app, which are not read.

Detail: JOURNAL.md.

3.6 Shared files being retired

These files are read and written by several programs today. Each has a replacement in the design:

File	Written today by	Replaced by
`data/industry-map.json`	qp's nightly SIC step (first), and each tool from TradingView's labels for gaps	the `sic` facts in the shared record, written only by the research worker
`data/oneil-market.json`, `oneil-groups.json`, `oneil-13f.json`, `data/oneil/`	qp's nightly chain	the O'Neil facts in the shared record, written only by the research worker (RESEARCH-WORKER.md)
`data/shortlist-all.json`	each tool, when its shortlist changes	home reads every tool's shortlist directly: in one program, every tool's file is at hand (HOME.md)

3.7 Secrets

File	Holds
`data/keys.json`	shared keys and the desk's switches: the shared record's address, the ntfy topics, the heartbeat address, the area modes, `workerShadow`, `deskTools`
`data/auth.json`	the password hash and the cookie's signing secret
`data/r2.json`	the backup bucket's keys
`data/journal/journal-accounts.json`	the journal's Alpaca accounts
`quant-platform/.env`	qp's feed keys, copied from the desk's by the deploy

The rules:

- mode 0600, gitignored;
- never in a backup, a log or a reply;
- set on the box with the commands in BOX.md §4.3.

3.8 Working files

Folder	Holds
`tmp/`, `tmp-t<n>/`	each tool's training CSV files and register exports; rebuildable
`data/worker/`	the comparison copies of tools being moved into the desk (`t<n>.db`), with their daily photo checks; deleted after the move
`src/scoring/outputs*`	the old scorer's model tables; no scorer runs; not carried (LEARNING.md)

---

4. Contracts

Writer	Writes only
the desk	each tool's own file (in that tool's context), the desk's own file, `data/tools.local.json`, `data/tool-stages.json`, the working files
qp	its own files and caches
the journal	its own file
the trader	the secrets, through the setting commands
the research worker	nothing on the box: its facts go to the shared record

5. When it fails

Event	What happens
a write fails half-way	the transaction is rolled back; nothing half-written is ever read
a file is damaged	the monthly rehearsal checks every SQLite file's integrity, and the restore puts back the last good night (BACKUPS.md)
the disk fills	writes fail loudly. The health page shows the free disk (to build); caches and working files are the first to go
a file is missing at start	the desk creates an empty one for a running tool, and says so. A stopped tool with no file is not served, and the health page says why: an empty history must never look like a tool that found nothing

6. Built on

The old desk.

- **The legacy data models:** screeners, qp and journal.
- **The code:** `src/db/index.js`, `src/toolsRegistry.js`, `quant-platform/chart/store.py`, `quant-platform/chart/alerts.py`.

What failed there:

- **The industry map had two writers and flipped every morning (F-21).** Fixed by letting SIC win; retired in the design.
- **Start-up repairs that silently did nothing (J-4).** Repairs now run once, recorded in `migrations`.
- **A capture-time change gave a job a new id and dropped its custom schedule (D-T6).** Jobs now have stable ids.

7. Status

Part	Status
each tool's own file, its tables and the registry	**built**
copies to the shared record, compared	**built** (step 9a; all eleven tools imported, 0 differences)
the live list saved after every change	**partly**: saved after every full scan; after every change with the move into the desk (step B)
`screener_versions` in each tool's file	**to build**
the desk's own file	**to build** (step B, with home and the desk jobs)
retiring the shared JSON files	**to build**, with the research worker switched on (step D) and home in the desk (step B)
dropping `analysis_model` and the dead settings	**to build** (a migration)

qp: charts, strategies, backtests, alerts

Above: ARCHITECTURE.md §4.4.

Talks to:

- the front door (`/quant/`);
- the shared record: registers, market states and facts, as of a day;
- the data sources: bars;
- the desk: its scheduler starts each alert cycle;
- the journal: the chart of a trade, and backtests exported as trades;
- its own files.

---

1. What it is

qp is the desk's chart and backtest program. It is the best-built part of the old desk, and its library, engine and backtest carry over almost unchanged.

Job	The question it answers
Chart	what does this stock look like, with indicators I can trust?
Build	can I describe this setup as rules, by clicking?
Backtest	if I had traded these rules on the stocks my tools found, on the days they found them, what would have happened, honestly?
Alert	is one of my saved strategies firing now on today's lists?
Print	let me review a day's (or a list's) charts, with the numbers behind them

What it owns:

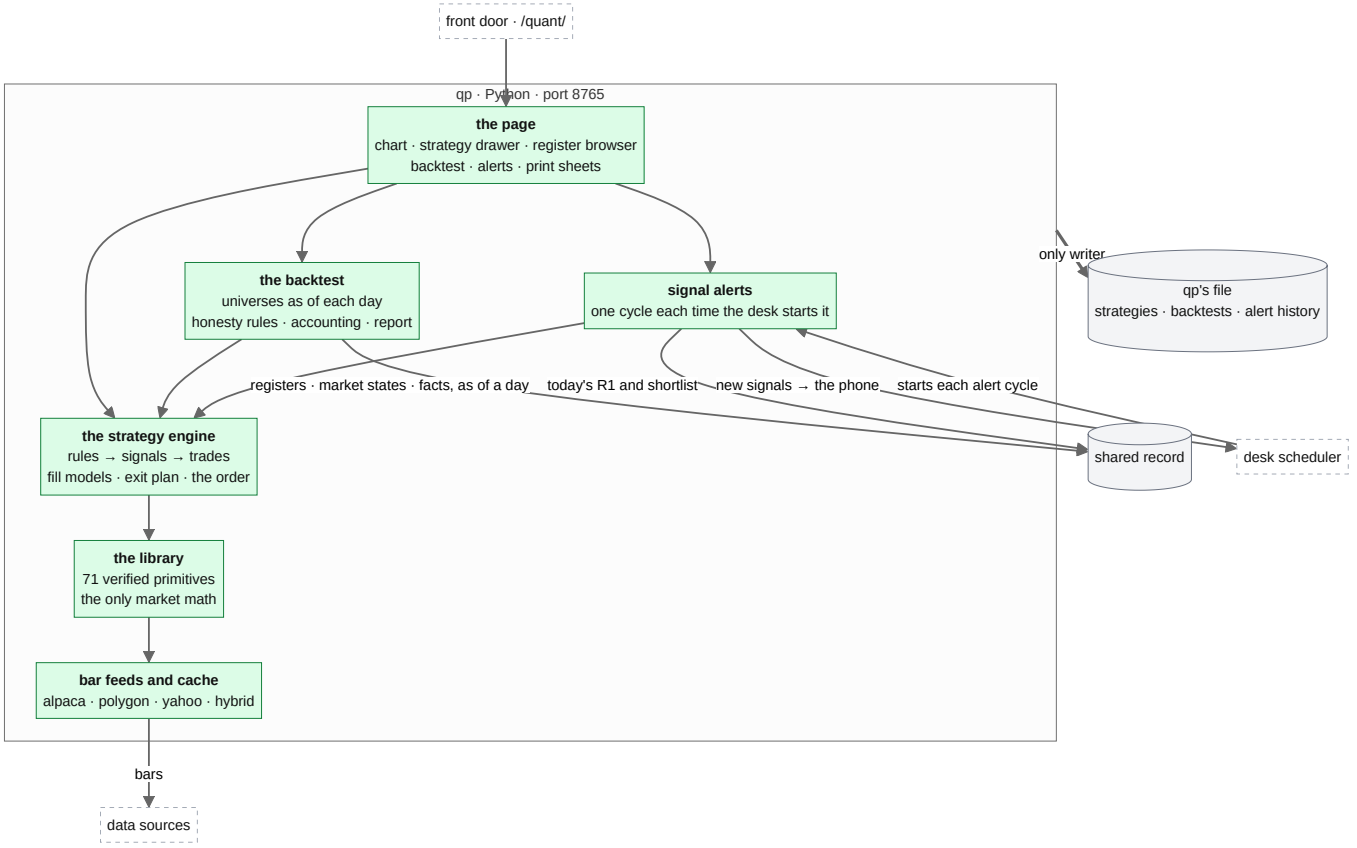
- strategies and their versions;
- backtests and their trades;
- the alert history;
- the library's approvals.

What it never does:

- compute a card fact or a market fact;
- keep a timer of its own;
- place an order: it alerts only.

---

2. The diagram



3. Its parts

Part	Job	Detail
the page	one page: chart, tools rail, indicators, the ⚡ strategy drawer (evaluate, test any rule, the signal funnel), the register browser, the backtest, alerts, print sheets	CHART.md
the library	the 71 verified primitives: the only market math anywhere in qp, each approved against TradingView	LIBRARY.md
the strategy engine	the strategy grammar, 14 operators, timing and groups; entries and exits; the risk block; the four fill models; the exit plan and the one order function	STRATEGIES.md
the backtest	the same evaluation as the preview, day by day over a universe as frozen that day; the honesty rules; settings and filters; accounting; the report and exports	BACKTEST.md
signal alerts	every saved strategy on today's lists, one cycle each time the desk starts it; one alert per strategy, symbol and bar; history kept; the cycle's new signals returned to the desk, which pushes them to the phone	SIGNAL-ALERTS.md
bar feeds and cache	the feeds (Alpaca, Polygon, Yahoo, the hybrid), warm-up, the bar cache and its limits, the borrow check	FEEDS.md

The working rule of the page: never trust a piece you have not seen. Plot it, test it, then compose it.

4. What changes from today

Change	Why	Detail
One fill model for preview, backtest and alerts. Alerts evaluate with the strategy's own fill (default `desk`)	an alert and a backtest of the same strategy must take the same bar (Q-N9)	SIGNAL-ALERTS.md
Registers, market states and facts are read as of a day from the shared record	today they are read over HTTP from each tool's port, and a stopped tool needed the archive program	BACKTEST.md
No O'Neil computation	the research worker owns it. While qp was down, every ticker first seen had a blank CANSLIM reading, and a photo then froze the blanks	RESEARCH-WORKER.md
No timer of its own	the desk's scheduler starts each alert cycle and records it, so a cycle that fails or does not run alerts	JOBS.md
Alert history kept	a restart never repeats an alert, and alerts can be reviewed	SIGNAL-ALERTS.md

Change	Why	Detail
Signals on the phone	today a signal reaches only an open page (a sound and a browser notification). The desk pushes each new one to its own ntfy topic; qp holds no push key	ALERTS.md §3.6
Shared bars per cycle	one fetch per symbol per cycle, shared by every strategy	FEEDS.md
The CANSLIM gate on the page	it is in the API only today (Q-N4); letters that cannot be rebuilt for a day are refused before the run	BACKTEST.md
`breakeven_after_target` simulated, or refused on save	it is reported and never simulated today (Q-N7)	STRATEGIES.md
Strategy versions	saving a strategy creates a version; a backtest records the versions it ran. The versions live in qp's own file	STRATEGIES.md
No per-route feed defaults	some routes default to Polygon, not the desk's default (Q-N6)	FEEDS.md

5. Contracts

The program: Python (FastAPI), `quant-platform/chart/server.py`, on port 8765 of the box, behind the front door at `/quant/`.

It owns (qp's file, `~/.qp/platform.db`)	Key
`strategies`, `strategy_versions` (versions: to build)	id; id, version
`backtests`, `backtest_trades`	id
`alert_seen`, `alert_history`	strategy, symbol, bar
the library's approvals	`approvals/approvals.json`, in git

It reads	From
bars	the data sources, through its feeds (FEEDS.md)
R1, the shortlist, R3, as of a day	the shared record (today: each tool's port; built but off: the desk's `/db/<tool>`)
market states, CANSLIM facts, as of a day	the shared record

Routes, behind `/quant/`	
the legacy routes carry over	53 routes and the live WebSocket (legacy/qp/API.md). The register routes read the shared record, and the alert routes read the stored history
`POST /api/alerts/cycle` (to build)	one alert cycle, started by the desk's scheduler; answers with the signals new in that cycle

6. When it fails

Event	What happens
qp is down	the desk is unaffected. Each alert cycle the scheduler starts fails, is recorded and alerts. Charts and backtests wait for qp to restart
a feed fails or returns nothing	the chart and the backtest say which feed and for which symbols; the backtest's coverage line counts the empty feeds
the shared record is down	backtests that need registers or facts say so and do not run; charts still work
one strategy or symbol fails in an alert cycle	the cycle goes on with the others; the error is in the cycle's record

7. Built on

The old desk: the whole of legacy/qp/:

- LIBRARY and CATALOG, with the library's code in library/;
- STRATEGIES and BACKTEST;
- ALERTS-AND-PRINTS and PAGE;
- DATA-FEEDS;
- SCALPS, SCALPS_SPEC and PINE;
- OPERATIONS and DATA-MODEL.

What failed there:

ID	Fault	Prevented by
Q-N9	alerts use the `close` fill and backtests `desk`, so an alert can fire on a different bar	one fill model
Q-N7	`breakeven_after_target` is reported, never simulated	simulated, or refused on save

ID	Fault	Prevented by
Q-N4	the CANSLIM gate has no control on the page	a control, reading facts as of the day
Q-N6	some routes default to Polygon	no per-route defaults
Q-N5	the nightly O'Neil chain lives in qp, on a hand-installed timer	moved to the research worker, started by the desk
—	registers are read over HTTP from every tool's port	read from the shared record
—	a qp restart took 3 min 20 s	fixed on 2026-10-06; one supervisor
—	alert repeats were held in memory	alert history stored (built, step 12)
—	each strategy-symbol pair fetched its own bars every cycle	shared bars per cycle
Q-N1–Q-N3, Q-N8, Q-N10	out-of-date help text	help generated from the definitions

The full list: legacy/qp/KNOWN-ISSUES.md.

---

8. Status

Part	Status
the library, engine, backtest, page, print sheets	**built** (legacy, kept)
alerts on each strategy's own fill model; alert history kept	**built, live** (step 12)
registers from the shared record	**built, off** (`quantRegistersMode`). Today qp reads each tool's port
facts and market states as of a day	**to build** (step D)
alert cycles started by the desk's scheduler, their new signals pushed to the phone	**to build** (step C). Today qp runs its own watcher thread, and signals reach only an open page
the O'Neil chain out of qp	**built** in the research worker, in shadow; the chain still runs from qp's timer until step C
strategy versions; the CANSLIM control; `breakeven_after_target`	**to build**
a re-run of past backtests against the shared record, compared	**to do**, before step D

The research worker

Above: ARCHITECTURE.md §4.5.

Talks to:

- the desk, whose scheduler starts every run (JOBS.md);
- the data sources: SEC EDGAR, Form 13F, Polygon, Yahoo;
- the shared record, where it writes its facts;
- the tools' files, which it reads, read-only, to train models.

---

1. What it is

The O'Neil research, done in one place. For every stock the tools find, and for the market as a whole, it works out what William O'Neil's method (CANSLIM) asks:

- how fast the company's earnings are growing (C and A);
- whether the stock leads: its strength against the whole market (RS), and its industry group's rank (L);
- whether it is breaking out of a sound base, or has something new (N);
- the supply of its shares (S), and whether big funds are buying (I);
- whether the market itself is in a confirmed uptrend (M).

It is a **job, not a program that stays up.** The desk's scheduler starts it; it works through its steps, writes each result with its date into the shared record, records its run, and **exits**.

It is the only writer of:

- every O'Neil fact: the letters above, and the ratings;
- the industry map: which industry each company belongs to, from its SEC filings (its SIC code);
- when a model is chosen, the tools' models.

What it never does:

- stay running;
- start on its own;
- make a page wait;
- write a tool's file.

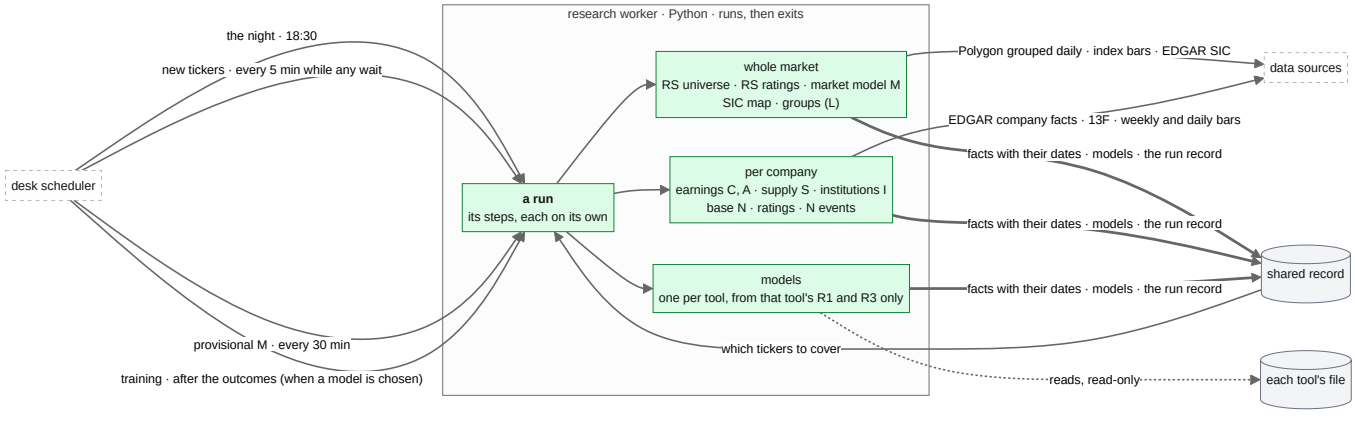
Why it is separate. Today the O'Neil research lives inside qp, the chart program, and each tool asks qp about each new stock:

- a stock first seen while qp is down or restarting gets a blank reading;
- a tool's morning record taken then keeps those blanks for good;
- five tools ask about the same stocks.

O'Neil results are card fields and Market-tab sections, so their writer must not depend on the chart program being up.

---

2. The diagram



---

3. Its parts

3.1 The runs

Run	When (New York, trading days)	Steps
the night	18:30, once the day's whole-market bars are published	every step, then the backup (BACKUPS.md)
new tickers	every 5 minutes, 04:00–16:00, while any ticker carded today has no company facts and no attempt today	the per-company steps, for those tickers only
provisional M	every 30 minutes in the session	the market model on today's index bars so far, written as **provisional**; the night's run replaces it
training	after the outcomes (about 16:20), when a model is chosen	one model per tool, from that tool's own history only (LEARNING.md)
by hand	a button, or a command	any step. It asks the scheduler to start the job; no page waits

New tickers the same morning.

- EDGAR allows ten requests a second, so a morning's new tickers land within minutes.
- Until they land, the card says "computing · since 09:41".
- RS, the group and the SIC code already exist for every US stock, from the whole-market steps.

3.2 The steps

Each step is independent. One failure never stops the rest, and each failure is reported with its reason.

#	Step	Writes (fact family)	Clock
1	RS universe top-up	`bars_universe` (Polygon grouped daily, the newest sessions)	each close
2	RS ratings	`rs_rating` per ticker, with the universe size	each close
3	market model	`oneil_m`: state, per-index counts, distribution days, rally, follow-through, the rules in use	each close; every 30 minutes in the session, provisional
4	SIC map	`sic` per ticker	nightly, for new listings and changes
5	groups	`group_ranking`; `oneil_l` per ticker (group, level, rank, percentile, letter, RS in group, rotation)	each close
6	earnings	`oneil_c`, `oneil_a` per ticker	weekly, and on a new 10-Q or 10-K
7	supply	`oneil_s` per ticker	monthly (shares); each close (U/D, A/D)
8	institutions	`oneil_i` per ticker	quarterly, after the 45-day filing deadline
9	base and RS line	`oneil_n_base`, `rs_line` per ticker	each close (weekly bars)
10	ratings	`oneil_ratings`: RS, group, A/D, volume against its 50-day average; later EPS, SMR, Composite	each close
11	N events	`oneil_n_events` per ticker	nightly, over 24 months
12	distribution days per stock	`oneil_dd` per ticker	each close

Which tickers each night:

- **Whole-market steps (1–5)** cover every US stock.
- **Per-company steps (6–12)** cover:
  - every ticker any tool carded in the last 90 days;
  - every R1 ticker;
  - every ticker in an open trade.

  A ticker carded for the first time is covered the same morning by the new-ticker run.

Which tickers to cover is read from the shared record:

- the desk records each carded ticker (`research_wanted`);
- the worker records each attempt (`research_attempts`).

Budgets. The per-company steps have a time budget per night. Whatever does not fit is covered on the following nights, oldest first.

The method behind each letter, its rules, thresholds and traps: ONEIL.md.

3.3 Facts, with their dates

Every fact it writes carries:

- **`as_of:`** the date it describes (the quarter end, the session, the 13F quarter);
- **`known_at:`** when the worker learned it;
- **`status:`** `ok`, `none` (no filings) or `unavailable` (EDGAR timed out), with the reason;
- **the rules in use**, for anything with a threshold, so a signal can be checked against its definition later.

So a backtest's CANSLIM gate reads exactly the letters known before its decision bar, and a photo records exactly the reading of its minute.

3.4 Who reads what

Reader	Reads	Never
the desk's card pipeline	the O'Neil facts of the card's ticker (CARD.md)	computes any of them
the desk's market reader	the latest `oneil_m`, final or provisional, and `group_ranking` (MARKET.md)	computes any part of them
qp	any of them, as of a day (BACKTEST.md)	writes them
the journal	the facts known at a trade's entry (REVIEW.md)	writes them
pages	through the card and the market state	trigger a build

3.5 Its switch

`researchMode` (in `data/keys.json`) has three settings:

- **shadow:** the cards keep qp's values, and the facts are compared beside them field by field.
- **on:** the cards read the facts; qp is asked only for tickers not covered yet.
- **off.**

---

4. Contracts

It owns (only writer), in the shared record	Subject
`bars_universe`, `group_ranking`, `oneil_m`	`MARKET`
`rs_rating`, `rs_line`, `sic`, `oneil_l`, `oneil_c`, `oneil_a`, `oneil_s`, `oneil_i`, `oneil_n_base`, `oneil_n_events`, `oneil_ratings`, `oneil_dd`	ticker
`research_attempts`, `research_runs`	—
models (when chosen)	tool, question, version

It reads	From
EDGAR (company facts, the ticker map, 8-K items, SIC), Form 13F datasets, Polygon grouped daily, Yahoo index and stock bars	the data sources (SOURCES.md)
the tickers to cover	`research_wanted`, written by the desk
R1 and R3 of a tool, to train its model	that tool's file, read-only

Its run record. Each run writes a `research_runs` row: its start and end, each step's result, and counts (tickers covered, facts written, failures with reasons). The scheduler's own run record holds the same outcome. A step that has not succeeded within its clock turns the health page amber, then red, and alerts. For example: earnings not refreshed in 8 days.

---

5. When it fails

Event	What happens
a step fails	the other steps run. The failure is recorded with its reason, and the facts keep their old dates, so cards say "last run DATE"
a run does not happen, or hangs	the scheduler records it as missed or hung, and alerts
a source is slow or limited (EDGAR: ten requests a second)	the step works within its budget and carries the rest to the next night
it runs out of memory at night	the run fails and alerts. The night's run happens at 18:30, when no scan is due, with swap behind it (BOX.md)
the shared record is down	nothing can be written, so the run fails and alerts. Nothing is written anywhere else

---

6. Built on

The code it runs: `quant-platform/research/worker.py`, reusing qp's O'Neil code, which was checked rule by rule: `relstrength.py`, `oneil.py`, `groups.py`, `edgar.py`, `f13.py`, `base.py`, `ratings.py`, `sic.py`.

The old desk:

- the O'Neil chain as built inside qp: legacy/qp/CANSLIM.md, 1,646 lines, the deepest document of the old desk;
- its nightly job: a systemd timer, Monday to Friday at 22:30 UTC, six steps (legacy/qp/OPERATIONS.md).

What failed there:

- The chain runs inside the chart program. Every tool asks qp for its new tickers' values and caches them for the day.
- A new ticker's earnings arrive only if a page shows its card, or the nightly walk reaches it.
- The nightly job runs from a hand-installed timer. Without it, the O'Neil data goes stale with no alarm (Q-N5).
- The industry map had two writers and flipped every morning (F-21).
- When an O'Neil file is missing, the Market tab asks qp to build it inside a page request.

---

7. Status

Part	Status
the worker: the O'Neil readings written as dated facts; new tickers within minutes; provisional M	**built, in shadow** (step 7). The cards still read qp's values
its runs started by the desk	**partly**: the new-ticker and provisional-M runs are started by T1's scheduler; the night's run is the last step of qp's own nightly job, on its systemd timer
the whole chain inside the worker (steps 1–12), started by the desk's night job	**to build** (step C): today the chain's computations run in qp's nightly job and the worker publishes their results
`rs_line`, `oneil_s`, `oneil_n_events`, `sic` as facts; the ratings beyond RS, group and A/D	**to build** (ONEIL.md §6)
training models	**prepared, no model** (by the trader's choice; LEARNING.md)

The journal

Above: ARCHITECTURE.md §4.6.

Talks to:

- the front door (`/journal/`);
- the desk, whose scheduler starts each account's sync;
- the data sources: Alpaca fills and orders, and bars;
- the shared record: the facts, the market state and the R1 row known at a trade's entry;
- qp: the chart of a trade;
- its own file.

---

1. What it is

Every real trade, after the fact: what was bought and sold, at what prices, in what size, against what stop, under what market, for what result. Then the patterns across trades: by hour, weekday, price, size, side and setup.

It answers the question the tools' records do not: **how do my actual trades go?** The tools' records measure what the screeners find; the journal measures what the trader did with it.

It is its own program, with its own file, because it shares nothing live with the tools.

What it owns:

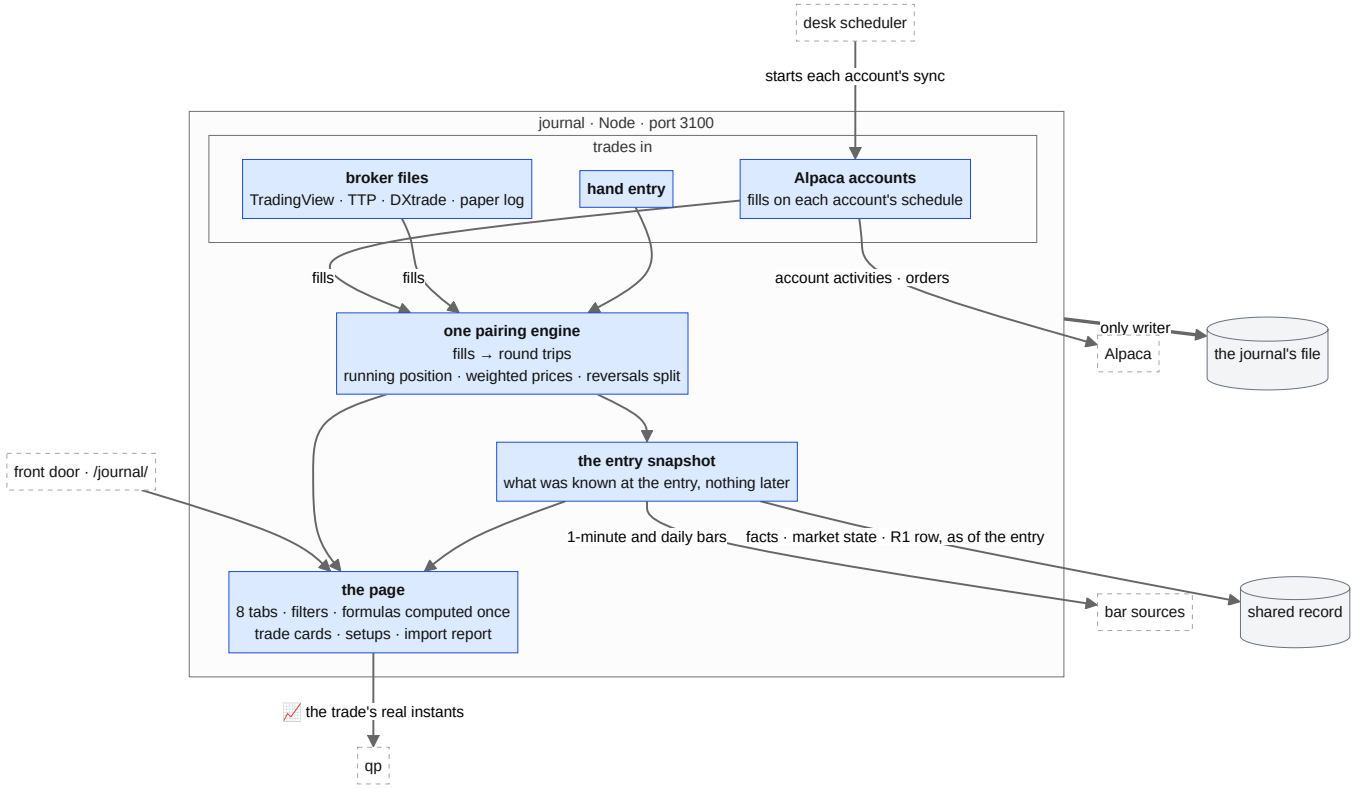
- the accounts;
- the fills;
- the trades, with their fills, stops, fees, setups and notes;
- each trade's snapshot at its entry;
- the day notes.

What it never does:

- place an order: the desk places none;
- grade or size a trade for execution;
- keep a timer of its own: each sync is started by the desk.

---

2. The diagram



3. Its parts

Part	Job	Detail
trades in	the three ways a trade arrives: Alpaca accounts synced on a schedule; broker files uploaded by the trader; hand entry. Each produces **fills**	IMPORT.md
the pairing engine	one engine, for every source, turns fills into round trips. It handles scale-ins, scale-outs, reversals and open positions once, with stable ids, so a second import updates and never duplicates	IMPORT.md
stops and fees	the stop from the broker's stop order, the strategy's exit plan, or the trader; fees from the broker, or from the account's fee rule (labelled estimated)	IMPORT.md
the entry snapshot	the price structure from bars up to the entry minute; the facts, the market state and the R1 row as known at the entry; MAE and MFE between the entry and the exit	REVIEW.md
the page	eight tabs (Overview, Calendar, Time, Risk, Stats, Setups, Trades, Import) and three filters; every number computed in one place; the trade cards; the setup statistics; the 📈 chart link	REVIEW.md

4. Contracts

The program: Node, `journal/server.js`, on port 3100 of the box, behind the front door at `/journal/`.

It owns the journal's file, `data/journal/journal.db`. The target tables are below. Today the trades live in `journal_trades`, with the setups, day notes and fee profiles; the old app's leftover tables are not carried (FILES.md §3.5).

Table	Key
`accounts` (keys in the secret file `data/journal/journal-accounts.json`)	id
`fills`	account, external id
`trades`	id (stable)
`trade_fills`	trade, fill, portion
`setups`	id
`trade_snapshots`	trade
`day_notes`	day

It reads	From
account activities (fills) and orders	Alpaca, per account

It reads	From
1-minute and daily bars	the bar sources (SOURCES.md)
facts, the market state and the R1 row, as of the entry	the shared record

Routes, behind `/journal/`	Do
`GET /api/journal/trades` (filters); `GET /api/journal/trades/:id`	trades; one trade with its fills and its snapshot
`PATCH /api/journal/trades/:id`; `POST /api/journal/trades`	stop, target, fees, setup, notes, tags; hand entry
`POST /api/journal/import`	a file in, an import report back
`GET/POST/DELETE /api/journal/accounts…`, `…/test`, `…/fetch`	accounts; no secret is ever returned
`POST /api/journal/sync` (to build)	one sync of the due accounts, started by the desk's scheduler
`GET/POST/PATCH /api/journal/setups`	setups
`GET /api/journal/stats` (filters)	every number of the page, computed in one place

5. When it fails

Event	What happens
an account's fetch fails	recorded with its reason; the account waits for its next turn; the sync's run record says which account failed
the journal is down	the desk is unaffected; the scheduler records the failed sync and alerts
a file has rows the parser cannot read	the import report lists every skipped row with its reason; nothing is dropped silently
the shared record is down	the snapshot waits and says so; trades still import

6. Built on

The old desk. legacy/journal/, documented from the code with every import format run on a scratch database:

- PAGE: every tab and formula;
- ACCOUNTS: the Alpaca accounts and their pairing;
- IMPORT: the file formats, and what was tested;
- ANALYSIS: the technical snapshot, conditions, grading;
- DATA-MODEL;
- OPERATIONS: start-up, the fetch timer, the tests;
- KNOWN-ISSUES: D-J1 to D-J20.

What failed there:

- **File imports mispaired scale-ins and scale-outs, lost TradingView shorts, and duplicated on re-import** (D-J1 to D-J7). Fixed by one engine (step 11).
- **No trade has a stop,** so every R is empty (D-J9).
- **Times were clock strings,** with a fixed −05:00 offset and one date per trade (D-J8, D-J10, D-J12). Fixed by real instants (step 4).
- **The snapshot and the grading half never worked** (D-J13, D-J14).
- **No fees for most sources** (D-J15).
- **The server's numbers differed from the page's** (D-J17).
- **None of the 84 journal tests covered the file importer.**

7. Status

Part	Status
the page and its formulas; Alpaca accounts on a schedule	**built** (legacy, kept)
real New York instants	**built** (step 4)
every broker file read as fills and paired by the one engine, with fees and a report of every row not read	**built** (step 11; D-J1 to D-J7)
stops; the entry snapshot from the shared record; MAE and MFE	**to build**
`accounts`, `fills`, `trade_fills` as tables; stable ids for files	**to build**
the sync started by the desk's scheduler	**to build** (step C). Today the journal runs its own minute timer
one condition system over the snapshot; setup statistics	**to build** (REVIEW.md)

Data sources

Above: ARCHITECTURE.md §4.7 (outside the box).

Talks to: every block that needs the outside world. Each one reaches it through adapters:

Block	Sources it uses
the desk	the scanner, quotes, short interest, borrow, news, index and sector quotes, outcome bars
the research worker	EDGAR, 13F filings, Polygon, index bars
qp	chart bars
the journal	account fills, bars

1. What it is

Every outside source the desk uses, and the **adapter** in front of each one.

An adapter is the only code that talks to one source. A provider that offers several kinds of data, such as Yahoo (quotes, charts) or Alpaca (bars, borrow), has one adapter for each kind. Each adapter keeps in one place everything about its source:

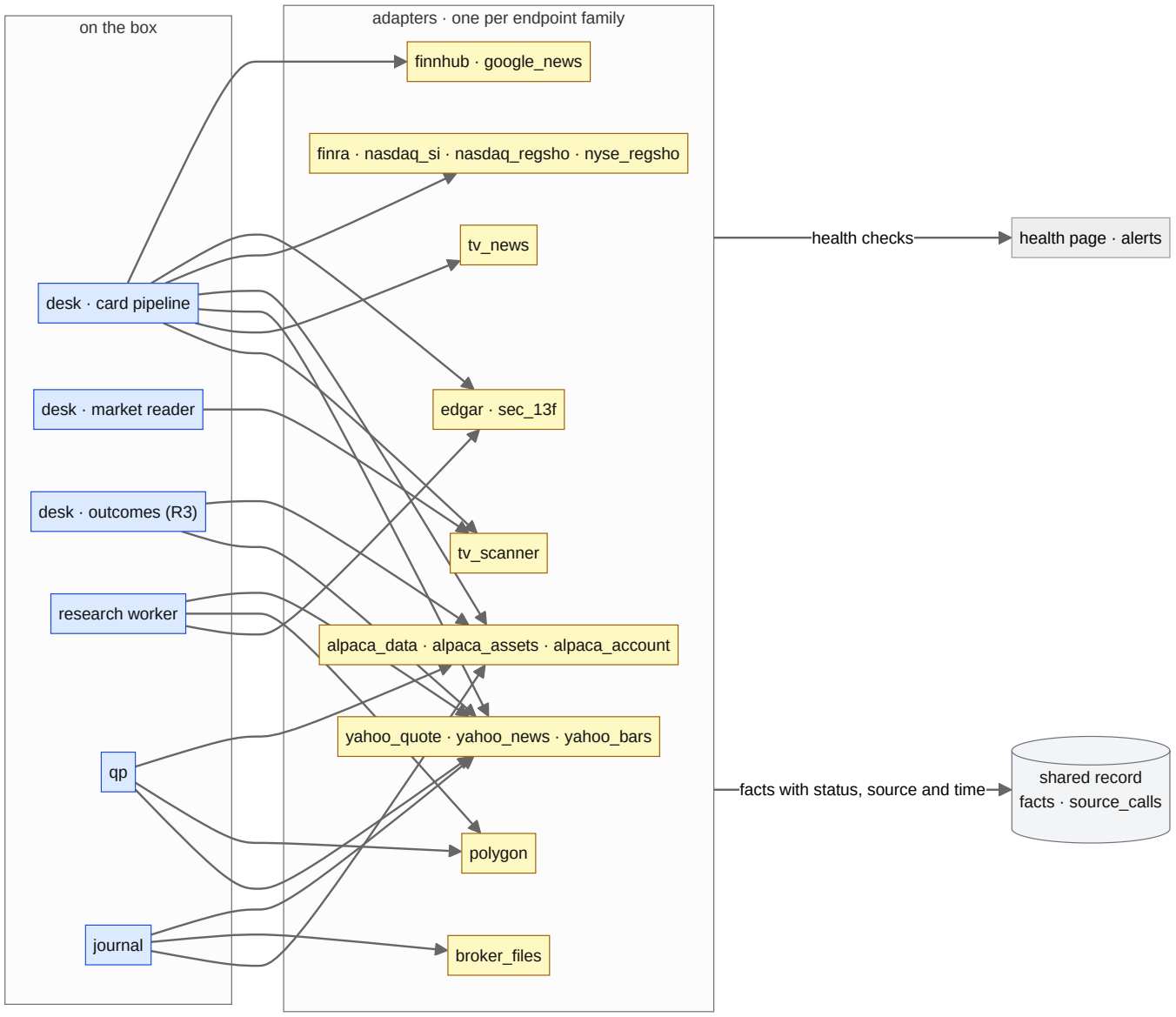
- how to ask;
- how fast it may be asked;
- how fresh its answers stay;
- how to read them;
- what "nothing" and "could not ask" mean;
- how its health is judged.

Every other block asks adapters for **facts**, and never calls a source directly.

What the adapters never do:

- guess a missing value (a missing value is `unavailable`, with the reason);
- read a missing value as 0, as "no" or as bearish;
- let a second source overwrite the first;
- let a source's failure fail a card.

2. The diagram



3. Its parts

3.1 The adapter contract

Each adapter states	Meaning
families	the fact families it can produce
request	endpoint, method, headers, timeout
auth	none, a key (named in the secret file), or a User-Agent
limits	requests per second or per day; batch size; time budget
freshness	how long a fact it produced stays fresh, per family
parse	turning the answer into fact values, with alignment and sanity checks
honest null	what `none` means (the source answered: nothing) and what `unavailable` means (could not ask), each with a reason
health	how its content is judged: ok, degraded, down, on a schedule
fixtures	recorded answers, including failures, for CI

Every call is counted. Each fact family keeps a count of the requests it received, the calls it actually made, its failures and its last error, per day (`source_calls`). These counts decide whether sharing a family across tools pays (SHARED-RECORD.md).

3.2 Every source

Adapter	Endpoint(s)	Auth	Families	Quirks
tv_scanner	`POST scanner.tradingview.com/america/scan?label-product=screener-stock`	none; a browser User-Agent, Origin and Referer of tradingview.com; 15 s	candidates per screener; `quote` (32 columns, batches of 200); index and sector-ETF quotes	**ignores unknown columns silently**, so the column list is verified. A batch whose rows differ in length from the columns asked is refused (D-S3). It warns when two range windows are identical on most rows. `short_percentage_of_float` is never filled. Longer-window highs are not split-adjusted
tv_news	`news-headlines.tradingview.com/v2/headlines`	none	`news`	—
yahoo_quote	`quoteSummary`	cookie and crumb from `fc.yahoo.com`	`short_interest` (fallback)	per symbol
yahoo_news	`query1.finance.yahoo.com/v1/finance/search`	none	`news`	—
yahoo_bars	chart API (1-minute, daily), with or without extended hours	none	bars	shallow 1-minute history (weeks); years of daily
alpaca_data	`data.alpaca.markets/v2/stocks/bars`, `/v1beta1/news`	key	bars (fallback), `news` (Benzinga)	the free plan's IEX feed: thin volume, about a week of intraday history, regular hours only
alpaca_assets	the assets API	key	`borrow` (shortable, easy to borrow)	**as of now only**, with no history: a backtest applies today's flag and says so
alpaca_account	`GET /v2/account`, `GET /v2/account/activities/FILL` (paper or live)	each journal account's own key, read-only	journal fills; key test	at most 50 pages × 100 fills per fetch
finra	`cdn.finra.org/equity/regsho/monthly/shrt{date}.txt`, then the OTC biweekly paths	none	`short_interest` (shares short; the card works out the % of float)	**one file for every symbol**, fetched once a day; settlement twice a month
nasdaq_si	`api.nasdaq.com/api/quote/{t}/short-interest`	none	`short_interest` (fallback, two weeks of history)	per symbol
nasdaq_regsho	`nasdaqth{yyyymmdd}.txt`	none	`reg_sho`	per settlement day
nyse_regsho	the NYSE threshold-securities download	none	`reg_sho`	per settlement day
edgar	`data.sec.gov/api/xbrl/companyfacts/CIK{…}.json`; `www.sec.gov/files/company_tickers.json`; submissions (SIC, 8-K items); full-text search	a descriptive User-Agent (403 without it)	`oneil_c`, `oneil_a`, shares for `oneil_s`, `sic`, 8-K events, `news`	ten requests a second; the five EDGAR traps (ONEIL.md); filings land weeks after the quarter's end
sec_13f	the SEC's quarterly Form 13F datasets	User-Agent	`oneil_i`	quarterly, 45 days after the quarter's end; managers over $100M only
polygon	grouped daily (the whole market); aggregates	key	`bars_universe`, bars	the free plan is end of day: no bars for today
finnhub	`finnhub.io/api/v1/company-news`	key	`news`	—
google_news	`news.google.com/rss/search`	none	`news`	asked only when the others return fewer than 2 items
openfda, **usaspending**	drug and device approvals; contract awards	none	N events (planned)	—
broker_files	uploaded TradingView, TTP and DXtrade files	—	journal fills	one parser per format, each with a sample file (IMPORT.md)

3.3 Fallback orders

Family	Order	Rule
`quote`	TradingView	a misaligned batch is refused; the last quote stays, with its age
`short_interest`	FINRA → Yahoo → Nasdaq	stop at the first plausible answer and record which source gave it. With no float, the % is of shares outstanding, and it says so
`borrow`	Alpaca	`unavailable` never means "not shortable"
`reg_sho`	Nasdaq + NYSE	both are asked; either listing counts

Family	Order	Rule
`news`	TradingView, Alpaca, Yahoo, EDGAR and Finnhub in parallel; Google when fewer than 2 items	the failing ones are named; the others still count
outcome bars	Yahoo → Alpaca	per ticker
chart bars	the chosen feed	never silently replaced (FEEDS.md)
index bars (M)	Yahoo	—
whole-market bars (RS)	Polygon	—

A second source for the same fact is stored beside the first, with the priority recorded. It never overwrites the first (the F-21 lesson).

3.4 Bars

Every consumer gets the same shape: the chart, strategies, backtests, alerts, outcomes and the journal all receive a UTC time-indexed frame of open, high, low, close and volume.

Feed	Source	Strength	Weakness
`yahoo`	Yahoo	free, current, years of daily	shallow 1-minute history
`yahoo_ext`	Yahoo with pre- and after-market	one source, more hours	—
`polygon`	Polygon	deep history, extended hours, consolidated tape	free plan: no bars for today
`alpaca`	Alpaca IEX	live	thin IEX volume, short intraday history, regular hours
`hybrid`	Polygon history + Alpaca today	deep and today	**a volume seam**: IEX volume is a fraction of Polygon's, which distorts every VWAP on the Alpaca part
`hybrid_yahoo`	Polygon history + Yahoo today	deep and today, both consolidated	—

Where the feed detail lives:

- choosing the feed, warm-up and the bar cache with its limits: FEEDS.md;
- outcome bars, stored once per ticker per day and shared by every tool: REGISTERS.md.

3.5 Health checks

The data check of the old desk asked each source for data that **cannot** legitimately be empty, and judged the content, but only when a button was pressed. In the design the same checks run **on a schedule**:

- each adapter at its own cadence;
- all of them before the day's first scan, at 03:45 New York.

They feed the health page (ALERTS.md).

Check	Down	Degraded
TradingView scanner: `close > 0`, 50 rows	zero rows; the first row has no usable price; the fetch failed	—
news: every source, for a liquid ticker	no stories from any	some sources failed, with the key named
Alpaca key: the account	rejected (401/403)	—
bar feeds: 40 daily bars each	the fetch failed; no data; under 15 rows; a close NaN or ≤ 0; every volume zero; the last bar over 5 sessions old; out of order or duplicated; a one-day move over 25 %	over 5 % of sessions without volume
indexes ^GSPC, ^IXIC	as bar feeds, with "M cannot be built"	—
cross-source agreement: the only check that catches a confidently wrong feed	the worst close difference over 0.5 %	under 3 shared sessions; fewer than 2 working feeds ("configure a second feed")
RS universe	no sessions; under 1,000 symbols	under 253 sessions
research facts	a required field missing	older than their clock allows, counted in trading sessions (M and groups: not from the last completed session; SIC: 14 days). The old check used 36 hours by the clock, which read every weekend as stale
EDGAR	the ticker map under 5,000; no quarters for a known filer; 403	—
sector map	—	a card sector with no ETF mapping (MARKET.md)

How checks are reported and run:

- Each failing check carries its **fix**.
- Checks run **inside the programs' own environment**, so they never report a different set of keys from the one in use. The first live data check did.
- A red check sends an alert; amber shows on the page.

3.6 Keys

Key	Used by
Alpaca (data)	`alpaca_data`, `alpaca_assets`
each journal account's Alpaca key	`alpaca_account`, per account
Polygon	`polygon`
Finnhub	`finnhub`
the EDGAR User-Agent	`edgar`, `sec_13f`

Where keys live, and what happens without one:

- All keys are in the secret files (BOX.md §4.3).
- A missing key makes its adapter `unavailable` with the reason "key not set", and the health page names it.
- Settings can set and test keys. The old Settings page had no inputs for them (D-T4).

---

4. Contracts

It owns	Where
`source_calls`: requests, calls, failures, last error, per family and day	the shared record
`health_checks`: each check's result over time (to build)	the shared record
the bar cache	qp's caches on disk (FEEDS.md)

What it serves. Facts to the desk, the research worker, qp and the journal, through one interface:

- **the call:** `fetch(family, subjects, as_of?)`;
- **each fact returned:** its status, source and the instant it was learned.

---

5. When it fails

Event	What happens
a source fails or times out	its families are `unavailable` with the reason; cards still build; the health check goes red and alerts with the fix
a source answers nonsense	the adapter's sanity checks refuse it; the cross-source check catches a plausible but wrong feed
TradingView changes or drops a column	the alignment check refuses the batch; the scan report says why; health goes red; no card gets shifted values
a free tier's limit is hit	the adapter stops at its limit and carries the rest; the limit is named on the health page
a key is missing or refused	the adapter is `unavailable · key not set` (or refused); nothing crashes

---

6. Built on

The old desk:

- DATA-PIPELINE: every call the card pipeline makes, with endpoints and fallbacks;
- DATA-FEEDS: qp's bar feeds;
- DATACHECK: judging each source by its content;
- CANSLIM §3.1 and §10.4: the free official sources for O'Neil data;
- ACCOUNTS: the journal's Alpaca calls.

What failed there:

ID	Fault	Prevented by
—	the same ticker's facts fetched once per tool	facts fetched once, shared
D-S1	short interest and borrow wiped on a re-quote	each family writes only itself
D-S2, F-14	a missing value read as a number or as bearish	the honest null
D-S3	a column shift misaligning every field	the alignment check

ID	Fault	Prevented by
Q-N6	routes with their own Polygon default	no per-route defaults
—	"alpaca ok" printed daily while the key was refused	health judged by content, on a schedule
—	the data check ran only when pressed, and once from the wrong environment	scheduled, inside the programs' environment
D-T4	the fix lines point to a Settings section with no inputs	keys set and tested in Settings
—	a full disk from an unbounded bar cache	the cache's limits

---

7. Status

Part	Status
the calls themselves, their fallbacks and the alignment check	**built** (legacy, inside the pipeline's stages)
short interest, borrow and news through the fact store, with call counts	**built, in shadow** (step 5)
one adapter per source for every fetch, with fixtures	**partly**; the rest **to build** as each family moves (step D)
the scheduled health checks and the health page	**to build** (step E)
keys set and tested from Settings	**to build**

The shared record

Above: ARCHITECTURE.md §4.7 (outside the box) and §5 (how the blocks share data).

Talks to:

Program	What it does here
the desk	writes the facts it fetches, and the copies of the registers and of the market history
the research worker	writes the O'Neil facts
qp	reads registers, market states and facts, as of a day
the journal	reads what was known at a trade's entry

The backups dump and verify it every night.

---

1. What it is

One database (Postgres) for what more than one program, or more than one tool, needs:

It holds	Why it is shared
facts about the world, each with its time and source	short interest, news, the CANSLIM letters and the rest are the same whichever tool asks. Stored once, with the instant they became known, they give every reader the same answer "as of" any instant
a copy of the market history: the market states and the hot sectors	the desk keeps the history in its own file; the copy lets qp read the market as of any day
the O'Neil facts	written by the research worker, read by the desk, qp and the journal
a copy of every tool's records: what it found each morning (R1), how those stocks moved (R3), its shortlist, its screeners and their versions	the copy keeps the research record safe if the box is lost between two nightly backups, and lets qp read any tool's history as of any day

It is not the record of a tool's results. Each tool's own file on the box is (FILES.md). So the desk keeps working when the shared record is down: the tools work from their own files and fetch their own facts, and the copies catch up.

What it never does:

- holds a secret;
- is written by a page directly;
- has two writers for one fact family or one table.

---

2. The diagram



---

3. Its parts

3.1 Facts

Table	Key	Holds
`facts`	subject, family, `known_at`	every fact: `value` (jsonb), `status` (`ok`, `none`, `unavailable`), `reason`, `source`, `as_of`, `provisional`, `tool`
`facts_latest`	subject, family	the newest fact of each subject and family (the same columns)
`source_calls`	family, day	requests received, calls actually made, failures, the last error
`fact_parity`	day, family, subject, tool	while a family is in shadow: the store's answer against the tool's own, and the fields that differ

What is kept.

- `facts_latest`: every fetch updates it.
- `facts`: keeps every fetch, except for the fast families: the quote, the numbers derived from it, and the market context stamped from each state. Those enter the history only at discovery, at each tool's photo and at the close.

So the history stays small. Every photo holds its whole card anyway, and the market history is kept on its own.

The families, their writers and their clocks:

- the card's families: CARD.md;
- the O'Neil families: RESEARCH-WORKER.md.

3.2 The copy of the market history

The desk keeps the market history in its own file (MARKET.md). These are its copies, written at the same moment and compared every day, plus the comparison of the shadow weeks.

Table	Key	Holds
`market_states`	id	`at`, `day`, `status` (`ok`, `no_regime`), `reason`, `state` (jsonb): the indexes, the three reads, the regime, the 15 sectors, the hot sectors, the rules in use
`hot_sector_days`	sector, day	each sector's best score of the day, its counts toward hot and toward cooling, hot or not, hot since
`market_parity`	tool, `at`	while the market reader is in shadow: each tool's own read against the shared one

Detail: MARKET.md.

3.3 Research

Table	Key	Holds
`research_wanted`	ticker, day	each ticker a tool carded, with when and by which tool (written by the desk)
`research_attempts`	ticker, day	each attempt of the research worker for a ticker, ok or not, with detail
`research_runs`	id	each run: kind, start, end, ok, each step's result, counts

3.4 The register copies

Table	Key	Holds
`tools`	id	name, status, role, capture times and why, the tradability floor
`screeners`	tool, key	name, status, mirror, purpose, current version
`screener_versions`	tool, key, version	the definition (jsonb), its hash, when, a note
`r1_photos`	tool, day, ticker	`captured_at`, the card (jsonb, with the versions of the screeners that matched), `late`, `source`
`r3_outcomes`	tool, day, ticker, entry	entry price, highest high, lowest low, ATR14, upR, downR, opening range high and low, `captured_at`
`shortlist_items`	tool, day, ticker	when added, by which method, the item (jsonb), when exported

How they are written and checked:

- The desk writes each copy at the moment it writes the tool's own file.
- The two are compared every day.
- The old history of all eleven tools was imported once, with 0 differences.

qp reads them as of a day (BACKTEST.md).

3.5 Run records

Table	Holds
`desk_log`	notable events, such as a hot-sector reset
`discovery_scans`, `engine_runs`, `engine_answers`	the discovery engine's comparison record. The engine is off and retires with the move into the desk (ARCHITECTURE §10)
`schema_migrations`	the migrations that have run

3.6 Its switches

Each area has a switch in `data/keys.json`:

Setting	Means
off	not used: each tool works as before
shadow	written and compared beside the tool's own way; nothing depends on it
on	used

Switch	Area	Today
`factsMode`	short interest, borrow, news from the fact store	shadow
`marketMode`	the one market read	shadow
`researchMode`	the O'Neil facts on the cards	shadow
`registersMode`	the register copies	shadow: written and compared, and it stays there
`discoveryMode`	the discovery engine	off
`quantRegistersMode`	qp reading registers from here	off

A family or an area is switched **on** only when its comparison is clean.

- **For a fact family**, also when its call counts show the tools share enough tickers for sharing to pay (MIGRATION.md step D).
- **The register copies have no "on".** They stay copies, and each tool's own file stays the record (ARCHITECTURE §10). What is switched on is qp reading them (`quantRegistersMode`).

3.7 Where it lives

On Railway today: a Postgres reached over TLS, its address kept in `data/keys.json` as `databaseUrl`.

The other places it could live:

- on the box, at 60–100 MB of its 912 MiB;
- on AWS RDS.

This is an open decision for the trader (ARCHITECTURE §10). Moving it is one dump and one restore.

What guards it:

- **Saving the address.** It is checked before it is saved: it must be a Postgres address, reachable from the box (not Railway's private address), and it must connect.
- **Its own database only.** Migrations refuse a database that already holds another project's tables.
- **One TLS rule** for every program that connects.

---

4. Contracts

Writer	Writes only
the desk	the card fact families, `source_calls`, `fact_parity`, the market tables, `research_wanted`, the register copies, `desk_log`, the discovery tables
the research worker	the O'Neil fact families, `research_attempts`, `research_runs`, the models (when chosen)
the backup	nothing: it reads, dumps and verifies

Reader	Reads
the desk	facts and O'Neil facts for its cards; the market state, only while the market history is not yet in its own file
qp	registers, market states and facts, as of a day
the journal	facts, the market state and R1 rows, as of a trade's entry

Migrations. They are numbered, run once and recorded, under an advisory lock so that two programs starting together cannot both migrate. Today they are `001_facts`, `002_market`, `003_research`, `004_registers`, `005_discovery` and `006_engine_answers`, in `src/facts/index.js`.

---

5. When it fails

Event	What happens
it is unreachable	every program stays off it for a minute, then tries again. Tools work from their own files and fetch their own facts. The copies and the shared facts wait, and the gap is shown on the health page
a write fails	the tool's own file already holds the row, so nothing is lost; the daily comparison finds the gap and the copy is written again
it is lost	restore last night's dump. The register copies can also be rebuilt from the tools' files (`scripts/import-registers.js`), with 0 differences proven once, and the market history from the desk's file

Event	What happens
the address saved is wrong	refused before saving, with the reason (BOX.md §4.3)

---

6. Built on

The old desk had no shared database:

- facts lived in each tool's memory, fetched once per tool;
- the O'Neil facts in JSON files written by qp;
- the market read was made five times a scan.

The design that made it: ARCHITECTURE §3.

The code:

- `src/facts/index.js` (the store, the migrations, the switches);
- `src/market/store.js`;
- `src/registers/store.js`;
- `quant-platform/research/worker.py`;
- the code repository's `docs/FACT-STORE.md` and `docs/REGISTERS-DB.md`.

What it learned:

- **Sharing a fact pays only when tools hold the same tickers.** The call counts decide each family.
- **The discovery engine saves nothing.** With every tool's screeners different, there is no identical scan to merge.

---

7. Status

Part	Status
the store, its migrations, its switches, the TLS rule, the foreign-database guard	**built**, on Railway (2026-10-08)
facts, market states, research facts	**built, in shadow**. The market states are today the only store; as a copy of the desk's own history they come at the end of step B
register copies, with the history of all eleven tools imported (0 differences)	**built**, written and compared
each area switched on	**to do**, area by area, when its comparison is clean (step D)
`health_checks`, models	**to build**
where it lives	**open**: the trader's decision

Backups

Above: ARCHITECTURE.md §4.7 (outside the box) and §8 (running it).

Talks to:

- the desk's jobs: the night job starts the backup after the night's research run, and the monthly job starts the rehearsal;
- the deploy (§4.2), which backs up before it restarts anything;
- every file on the box, which it reads;
- the shared record, which it dumps;
- Cloudflare R2, where everything goes;
- the alerts, for a night that is not clean.

---

1. What it is

A copy of everything the desk records, off the box, every trading night, proven restorable.

- **The backup** copies every file on the box and every table of the shared record to cloud storage (a Cloudflare R2 bucket). It then proves the copy of the shared record by restoring it into a scratch area and counting the rows.
- **The restore** puts any kept night back with one command.
- **The rehearsal** restores the newest night every month, so the restore is known to work before it is needed.

It also leaves **analysis files** in the bucket: each tool's records as one file per day, which analysis tools such as DuckDB or pandas read in place.

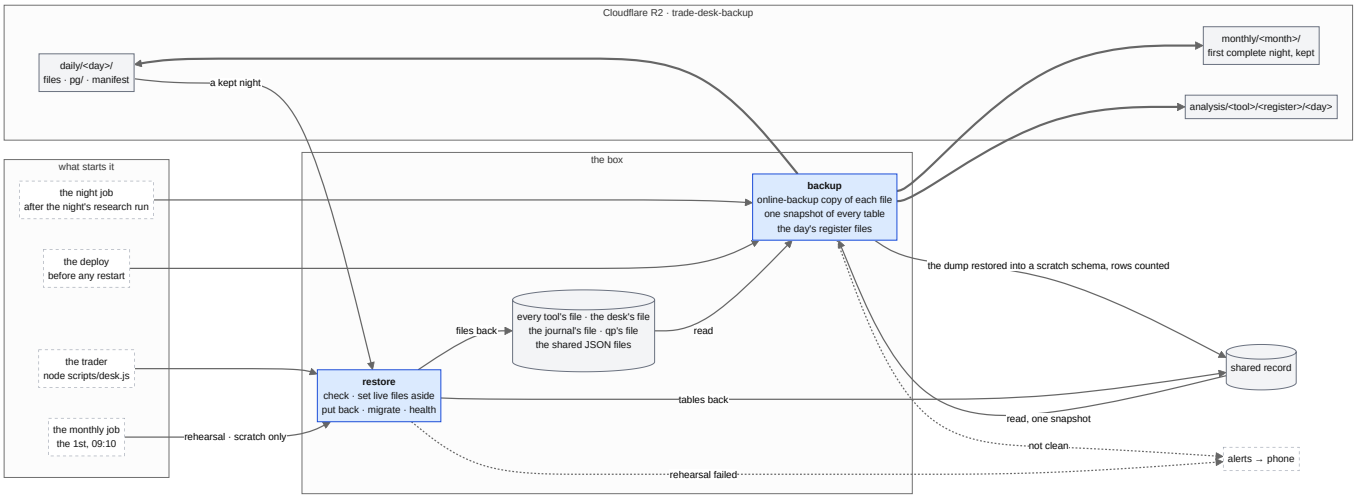
What it never does:

- copies a secret;
- writes over a live file without moving it aside first;
- counts a night as kept when anything in it is missing.

Why it matters. The box is one machine with no second copy. Each tool's file is the record of its research, and a lost file is a lost history. The shared record's copy of the tools' records covers the hours since the last night (SHARED-RECORD.md); the backup covers everything else.

---

2. The diagram



---

3. Its parts

3.1 When it runs

Trigger	When	Why
the night	each trading night, as soon as the night's research run ends, whatever its result	a dependency, not a clock time: the backup holds the night's facts. Tonight's registers are worth keeping even on a night the research failed

Trigger	When	Why
the deploy	before it restarts any program	a restart is when migrations run, so there is always a copy from just before a schema changed
by hand	`node scripts/backup-r2.js` (`--dry-run` builds everything and uploads nothing)	

3.2 What it copies

Source on the box	In the bucket
every SQLite file in `data/`: each tool's file, running or stopped (T1's is `tradedesk.db`), and the desk's own file	`data/<same>.db.gz`
the journal's file, `data/journal/journal.db`	`data/journal/journal.db.gz`
qp's file, `~/.qp/platform.db`	`qp/platform.db.gz`
the shared JSON files in `data/` (the tool registry, and the files being retired, FILES.md), but no secret file	`data/<same>.json.gz`
every table of the shared record, read in **one snapshot**	`pg/<table>.jsonl.gz`, JSON lines; no `pg_dump` is needed on the box

Consistent copies. Each SQLite file is copied with SQLite's online backup API, so a copy taken while the desk is writing is still a whole database, never half a write. The shared record is read in one transaction, so its tables agree with each other.

Stopped tools are included. Their files are the only copy of what they collected.

3.3 What it never copies

File	Why
`data/keys.json`	the API keys and the shared record's address
`data/r2.json`	the backup's own credentials
`data/auth.json`	the login's password hash and signing secret
`data/journal/journal-accounts.json`	the journal's Alpaca account keys

A backup must not be the place a key leaks from (S-2). The trader keeps their own copy of the secrets, and a restore puts them back from it (BOX.md §4.3).

Not copied, because they rebuild themselves:

What	How it comes back
the bar caches (`~/.qp-cache/`)	fetched again on demand
trained score models (`src/scoring/outputs*`)	trained again from the registers. When a model is chosen, models become rows in the shared record, and the dump carries them (LEARNING.md)
the comparison copy's file (`data/worker/`)	disposable: it is rebuilt from the tool's file
logs, qp's operations log	logs, not records
the code	the repository

3.4 The bucket

```
trade-desk-backup/
├── daily/<day>/                    one folder per night, by New York date
│   ├── manifest.json               what was copied, sizes, sha256, failures, the check
│   ├── data/<file>.db.gz           every SQLite file
│   ├── data/<file>.json.gz         the shared JSON files
│   ├── data/journal/journal.db.gz
│   ├── qp/platform.db.gz
│   └── pg/<table>.jsonl.gz         every table of the shared record
├── monthly/<month>/                the first complete night of each month, kept
└── analysis/<tool>/<register>/<day>   one file per tool, register and day
```

The manifest. One per folder: the date, the start and end, the host, each file with its kind, bytes and sha256, the register files with their row counts, the shared record's tables with their row counts and the check's result (`pg.verified`), `failures` (anything that should have been copied and was not) and `notes` (not failures: a register with no rows that day).

The check. Before a night counts as kept, the dump is restored into a scratch schema of the shared record and each table's rows are counted against the live counts read in the same snapshot. A mismatch fails the night.

A month is kept only from a night with zero failures. The month's own manifest is written last, and only then. While it is missing, each night writes its copy to the month again. So a failed night is retried the next night, and the month is never marked kept with a file missing.

Retention is set on the bucket, not in code: a lifecycle rule deletes each `daily/` folder after 60 days. `monthly/` and `analysis/` are kept.

Size. About 17 MB compressed a night (measured 2026-10-02).

3.5 The analysis files

Each tool's registers for the day, one file per tool, register and day, written as the registers are defined in REGISTERS.md.

- **Format:** Parquet, so the columns keep their types and DuckDB and pandas read them in place. Today they are gzipped CSV, nested values as JSON in the cell.
- **How they are read:** through the desk's register code, with each tool's file opened read-only. A running tool, a stopped one and a desk that is down all export the same way, and the same answer the pages and qp get.
- `node scripts/backup-r2.js --all-dates` exports every past day, after a restore or a tool's history import.

```
SELECT * FROM read_parquet('analysis/T2/R1/*.parquet');
```

3.6 The restore

One command, `scripts/desk.js`, for the files and the shared record together:

Command	Does
`node scripts/desk.js restore --list`	the nights and months kept, each with its manifest's result
`node scripts/desk.js restore <day> --to scratch`	the shared record into a scratch schema and the files into `restore/<day>/`; nothing live is touched
`node scripts/desk.js restore <day> --yes`	puts that night back on this box. Without `--yes` it only says what it would do
`node scripts/desk.js restore monthly/<month> --yes`	the same, from a monthly copy
`node scripts/restore-r2.js <day> [file | all]`	download only, into `restore/<day>/`

A live restore, in order:

1. checks every download against its sha256, and every SQLite file's integrity;
2. stops the programs (the desk, qp, the journal);
3. moves each live file aside to `<file>.pre-restore-<time>`, never deleting it, and dumps the live shared record to `restore/pre-restore-<time>/pg/`;
4. puts the night back;
5. runs the migrations;
6. starts the programs and checks each one's health;
7. prints each table's rows against the manifest.

3.7 The rehearsal

On the 1st of each month at 09:10 New York, the monthly job restores the newest complete night into the scratch schema, counts every table and checks every SQLite file's integrity. A failure is a red alert. By hand: `node scripts/desk.js rehearse`.

3.8 A new box

1. Make the box and deploy: `./deploy-tools.sh` starts empty (BOX.md).
2. Put the secrets back from the trader's copy: `scripts/set-r2-keys.sh`, the shared record's address, `scripts/set-password.sh`.
3. `node scripts/desk.js restore <day> --yes`.
4. Put the API keys back (`scripts/set-setting.js`, `scripts/set-alpaca-keys.js`) and the journal's accounts.

Nothing else is copied by hand.

The proof, once: a fresh box, restored with one command, serves the same pages and numbers as the old one for a chosen past day.

3.9 Its credentials

`scripts/set-r2-keys.sh` asks for the R2 endpoint, the access key id, the secret (typed hidden) and the bucket. It writes `data/r2.json` at mode 0600 and tests the keys with a read-only request.

- **The key:** a Cloudflare account API token scoped to this bucket, with object read and write.
- **To rotate:** create a new token, run the script again, delete the old one.

---

4. Contracts

It reads	It writes
every file in §3.2, read-only; the shared record, in one snapshot	the bucket; the scratch schema of the shared record, for the check; its run record, through the desk's jobs

The restore reads	It writes
a kept night or month	`restore/<day>/`; with `--yes`, the live files and the shared record, after setting the old ones aside

Its run record. The night job and the monthly job record each run in the desk's job history (JOBS.md): the files and bytes copied, the register files, the check's result, the failures. A night that is not clean fails the job, and the job alerts.

---

5. When it fails

Event	What happens
a file cannot be copied	the night is recorded with that failure and alerts. The month is not marked kept, so the next night tries again
the dump does not verify	the night fails and alerts; the previous kept night stands
R2 is unreachable, or the keys are wrong	the night fails and alerts with the reason. The files on the box and the shared record's copy of the registers are untouched
no `data/r2.json`	the backup is off, and says so in the night's record and on the health page
the rehearsal fails	a red alert. Until it passes, the restore is not known to work
a restore stops midway	the files set aside (`<file>.pre-restore-<time>`) and the live shared record's dump are kept. Putting them back is the same restore, from the set-aside copies
the box is lost	§3.8. Photos taken since the last night are in the shared record's copy of the registers

---

6. Built on

The old desk.

- **A GitHub backup** pushed the API keys in plain text (S-2). It is gone.
- **Then a nightly R2 backup** under a pm2 cron, weekdays 23:20 UTC and once at every deploy. It copied every file but kept no history, did not alert, and ran fifty minutes after the O'Neil chain started, whether or not the chain had finished.
- **Restore was manual:** download, stop each program by hand, copy each file, delete its journal files, start again. A full restore onto a box had never been done.

The design that made it: the old PLATFORM §5–6, now carried here.

The code:

- `scripts/backup-r2.js`, `src/backup/r2.js`, `src/backup/pg.js`;
- `scripts/desk.js` and `src/backup/restore.js` (restore and rehearsal);
- `scripts/restore-r2.js`, `scripts/set-r2-keys.sh`;
- `src/jobs/night.js` (the night: the chain, then the backup);
- `tests/backup.r2.test.js`, against a fake S3 server;
- the code repository's `docs/BACKUPS.md`.

---

7. Status

Part	Status
the nightly backup, the verified dump, the monthly copies, the one-command restore, the monthly rehearsal	**built** (step 13)
started by the desk's night job after the research run	**built**. It runs from the old pm2 cron until `scripts/retire-old-schedulers.sh` hands it over (step C)
the backup before every deploy	**to build** (step C). Today the pm2 cron runs it once at each deploy, beside the restarts rather than before them
the analysis files read through the desk with each file opened read-only	**to build** (step A, when the archive stops). Today each tool's port and the archive serve them
the analysis files as Parquet, under the registers' new names	**to build**. Today gzipped CSV, under the old names (R3A, R3B, R4A, R4B)
three monthly rehearsals passed	**to do**. Until then, the old manual steps stay written down

Part	Status
the new-box test	**to do**, once, before the old manual steps retire

Alerts and the health page

Above: ARCHITECTURE.md §4.7 (outside the box), §6.1 (when something goes wrong) and §7 (the rule "failures are loud").

Talks to:

- the desk's jobs: every run's result, and the jobs that check sources, facts, programs and the box;
- the data sources §3.5: the scheduled health checks;
- the shared record: fact freshness, and whether it can be reached;
- the backups: a night that is not clean, a failed rehearsal;
- qp: the signals its alert cycles find (SIGNAL-ALERTS.md);
- the box: memory, disk, restarts;
- ntfy, which delivers to the phone, and an outside heartbeat check;
- the desk's home, which serves the health page.

---

1. What it is

How the desk says something is wrong, and what is right.

- **The signals:** every way the desk can fail, each judged in one place, with its state (green, amber, red), since when, and the fix.
- **The phone:** a red signal is pushed once, through ntfy, and its recovery is announced once.
- **The health page:** every signal on one page, with its fix.
- **The heartbeat:** an outside check that alerts when the desk itself stops reporting, because a desk that is down cannot say so.

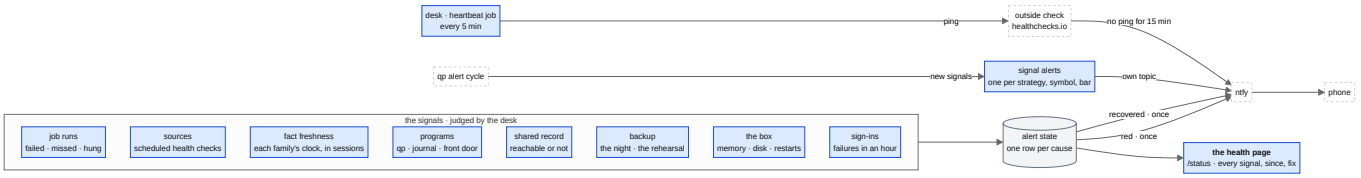
It also carries qp's signal alerts to the phone: a saved strategy firing on one of today's stocks. Those are trading signals, not faults, so they arrive on their own channel.

What it never does:

- repeats an alert while its cause stands;
- stays silent when a job fails or does not run: a silent no-op is a bug;
- sends a secret, a key or a stack trace to the phone;
- lets qp or the journal push on their own: the desk is the only sender.

---

2. The diagram



---

3. Its parts

3.1 The signals

Each signal has one judge, and amber and red mean the same everywhere:

- **green:** as it should be;
- **amber:** shown on the health page, not pushed. Something to look at;
- **red:** pushed to the phone once, and shown on the health page until it clears.

Signal	Judged by	Amber	Red
a job's run	the scheduler, on every run (JOBS.md)	one failed run of a job that runs more often than every 15 minutes (a scan, a re-quote, the new-ticker run, an alert cycle, a journal sync)	any other job failed, missed or hung; a frequent job failed three runs in a row
a source	the scheduled health checks (SOURCES.md §3.5), before the day's first scan and at each source's cadence	degraded: some of it failed, or a free tier's limit was reached	down: rejected, empty where it cannot be, or no answer

Signal	Judged by	Amber	Red
a fact family's freshness	the desk's freshness job, over `facts_latest`, against each family's clock (CARD.md, RESEARCH-WORKER.md)	older than its clock	older than twice its clock
a program	the desk, asking each program's `/health` every minute	—	qp, the journal or the front door not answering for 3 minutes
the shared record	every program that uses it	unreachable: the tools work from their own files, and the copies wait	unreachable for an hour, or the daily comparison of the copies finds differences
the backup	the night job and the monthly job (BACKUPS.md)	the backup is off (no `data/r2.json`)	a night not clean; the dump did not verify; the rehearsal failed
the box	the desk, every 5 minutes	memory available under 100 MB for 5 minutes; disk under 20 % free	disk under 10 % free; a program restarted by its supervisor (a crash or its memory limit)
sign-ins	the front door (FRONT-DOOR.md)	—	10 failed sign-ins within an hour
the desk itself	the outside heartbeat check (§3.5)	—	no heartbeat for 15 minutes

Counted in trading sessions. A clock that spans a weekend or a holiday is counted on the trading calendar, so Monday morning never reads as stale. The proof: on a Monday before the open, and on the morning after a holiday, no freshness signal is amber.

While a tool moves into the desk (MIGRATION.md step B), the comparison copy also sends one line a day: that day's photo comparison, clean or not.

3.2 The alert state

One row per cause, keyed by what failed (`job:<job>`, `source:<adapter>`, `fact:<family>`, `program:<name>`, `box:disk`, …).

Column	Holds
`key`	the cause
`failing`	1 while it stands
`since`	when it started, or when it recovered
`message`	what failed, and the fix
`sent`	whether the phone was reached (to build)

The rules:

- **Raised once.** The first red raises it and pushes. While it stands, nothing more is sent.
- **Recovered once.** The first good result after it clears it and announces the recovery.
- **Delivered, or tried again.** A push that did not arrive (ntfy unreachable) is tried again at the next judgement while the cause stands (to build: today it is recorded and not tried again).

Where it is kept. A tool's causes are kept in that tool's file; the desk's own causes in the desk's file (FILES.md). The health page reads them all.

3.3 The message

Part	Holds
title	who and what: `T4 · R1 Capture 09:36 failed`, `desk · Polygon down`, `desk · recovered · the night`
body	what failed; since when, in New York time; **the fix**, or where to look (the run's record)
priority	red: high. A recovery or a test: default

Never in a message: a key, a password, an address with credentials, or a stack trace. The stack stays in the log and in the run's record.

3.4 Delivery: ntfy

Setting (`data/keys.json`)	Holds
`alertNtfyTopic`	the topic for faults: a long name only the trader knows
`signalNtfyTopic`	the topic for qp's signal alerts (to build)
`alertNtfyServer`	the ntfy server; `https://ntfy.sh` when not set

The topic is a secret. Anyone who knows it can read the alerts, so it is set with `scripts/set-setting.js` and kept like a key.

Two topics, two channels. Faults and trading signals arrive on different topics, so the trader can silence one without missing the other.

Without a topic, an alert is still written to the log, the alert state and the health page, and the health page says that nothing reaches the phone.

A test: `POST /api/monitor/alerts/test` sends one test alert.

3.5 The heartbeat

The one alert that does not come from the desk, because it is about the desk.

- Every 5 minutes, a desk job pings an outside check (healthchecks.io, free).
- When no ping arrives for 15 minutes, the outside check alerts the phone through its ntfy integration, on the faults topic.
- When the pings resume, it announces the recovery.

What it catches: the desk down or restarting in a loop; the desk's scheduler stuck (the ping is itself a job); the box down; the box's network down.

Its address (`heartbeatUrl` in `data/keys.json`) is a secret like the topic.

3.6 Signal alerts to the phone

When the desk starts one of qp's alert cycles, qp answers with the signals that are new in that cycle (SIGNAL-ALERTS.md).

- The desk pushes each one to the signals topic: the strategy, the symbol, the bar, the price.
- qp's alert history already holds one row per strategy, symbol and bar, so a signal is never pushed twice, even across a restart.
- The page still sounds and notifies, as today.

3.7 The health page

One page, served by the desk's home at `/status`, phone first. It shows every signal of §3.1, grouped:

Group	Shows
the desk	up since; the heartbeat; the last scan of each running tool
jobs	each job of each tool and of the desk: its last run, its result, its next slot, a link to its run history
sources	each adapter's last check and its result, with the fix
facts	each family's age against its clock
programs	qp, the journal, the front door: answering or not, and their version
shared record	reachable or not; how far the copies are behind; the last comparison
backup	the last night, its check, the last rehearsal
box	memory, swap, disk, restarts
delivery	whether alerts reach the phone; a button that sends a test

Each red or amber line says **since when** and **the fix**. The page asks the desk every 30 seconds.

---

4. Contracts

It owns	Where
`alert_state`	each tool's file (its causes); the desk's file (the desk's causes)
`health_checks`, each check's result over time	the shared record (SOURCES.md)
`alertNtfyTopic`, `signalNtfyTopic`, `alertNtfyServer`, `heartbeatUrl`	`data/keys.json`, mode 0600

It reads	From
every job's runs	`job_runs`, in each tool's file and the desk's file (JOBS.md)
fact ages	`facts_latest`, in the shared record
each program's health	its `GET /health`
new signals	the answer of each alert cycle

Route (served by the desk)	Does
`GET /status`	the health page
`GET /api/status`	every signal, with its state, since when and its fix
`GET /api/monitor/alerts`	the causes failing now, and whether alerts reach the phone
`POST /api/monitor/alerts/test`	one test alert

Only the desk pushes. qp and the journal never send an alert; their faults are seen by the desk's jobs that start them and ask their health.

---

5. When it fails

Event	What happens
ntfy is unreachable	the alert is in the log, the alert state and the health page; the push is tried again while the cause stands
no topic is set	nothing is pushed; the health page says so in red
the desk is down	no alert can come from it. The heartbeat check alerts within 15 minutes
a check itself fails (an error in the judge)	that signal turns amber with the error, never green
a cause flaps (fails, recovers, fails)	each change is one push. A job that flaps is itself the fault to fix

---

6. Built on

The old desk had no alerts:

- the data check was one button, up to two minutes, judging each source's content only when pressed (legacy/screeners/DATACHECK.md);
- qp's `feedcheck` ran at deploy, and qp's run scripts from a shell;
- a nightly job could fail every night unseen (Q-N5), and the journal's fetch could fail silently (J-4);
- "alpaca ok" was printed daily while the key was being refused.

The design that made it: the old PLATFORM §4, now carried here. qp's signal alert to the phone comes from the old QUANT design ("a push to the phone as well as the page").

The code:

- `src/alerts/push.js`: ntfy, raise once, recover once, the alert state;
- `src/scheduler.js`: failed, missed and hung runs;
- `src/routes/monitor.js`: the alert routes;
- `src/worker/parity.js`: the comparison copy's daily line.

---

7. Status

Part	Status
ntfy to the phone; raised once, recovered once; the alert state; the test route	**built**, tested on the phone 2026-10-08
failed, missed and hung runs alert	**built** for R1, R3, the night and the rehearsal. **To build:** every job, with the frequent jobs' three-in-a-row rule (step C)
the comparison copy's daily line	**built** (step 15b)
the scheduled source checks, fact freshness, program health, the box, sign-ins	**to build** (step E)
the health page, `/api/status`	**to build** (step E)
the heartbeat	**to build** (step E)
signal alerts to the phone, on their own topic	**to build** (step C, with the alert cycles started by the desk)
a failed push tried again	**to build** (step E)

From today's desk to the design

Above: ARCHITECTURE.md §9 (where the build stands, and the way there).

Talks to: every block's document. Each step below names the blocks it changes, and each block's status table names the step that finishes it. The commands on the box are in the code repository's `docs/SETUP-NOW.md`.

---

1. What it is

How the desk that runs today becomes the design, without a rewrite and without stopping the desk.

It has two halves:

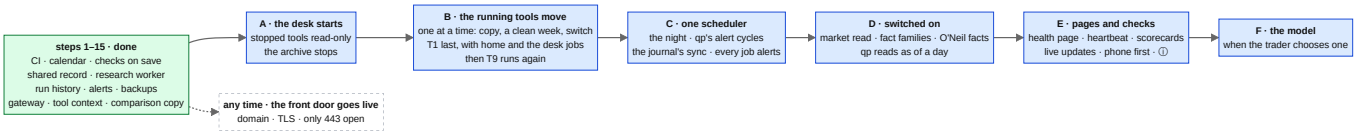
- **The data half, steps 1–15, is done.** It gave the desk its shared record, the research worker, run history and alerts, the backups, the one address, the way one program holds many tools, and the side-by-side comparison of a tool. Most of it runs in shadow: written and compared, not yet used.
- **The program half, steps A–F, is next.** It moves every tool into the one desk program, gives the desk the only scheduler, switches on what the comparisons have earned, and adds the pages.

What it never does:

- stops the desk to move a part;
- switches a part off before what replaces it has matched it;
- loses a row of any tool's research record.

---

2. The diagram



---

3. Its parts

3.1 The rules

1. **The old part keeps running** until the new one has matched it for a set period: a week of trading days, unless a step says otherwise.
2. **Compare, then switch.** Every step has a check against what it replaces. Each difference is explained (a fault the new part fixes) or fixed.
3. **One step at a time**, each a pull request through CI.
4. **Reversible.** Each step says how to go back.
5. **The research record is sacred.** No R1, R3 or shortlist row of any tool, running or stopped, is lost. Every import is checked by row counts.
6. **Room first.** Nothing starts on the box without the memory for it.
7. **Something visible.** Each step ends with something the trader can see.

3.2 The order, and why

The box has no memory to spare. On 2026-10-06 the programs used about 724 MB beside the system: five tool programs at about 90 MB each, qp, the archive and the journal. The gateway has been added since. A comparison copy run as its own program (step 15b) costs about as much as the tool itself, and there is no room for it.

So the desk starts where it frees, or at least costs, nothing:

- **A.** The desk takes the archive's job: the stopped tools, read-only. It costs about what the archive costs, and it proves the desk's web layer, contexts and read-only files on tools whose data cannot change.
- **B.** Each running tool is then compared as a **context** inside the desk, tens of megabytes rather than a program. Each tool that moves frees its program's 90 MB, which makes room for the next comparison.
- **C** comes after B, because the desk jobs live in the desk only once T1 has moved.
- **D's switches** can each start as soon as its comparison is clean; most will come after C, once the whole O'Neil chain runs in the research worker.
- **E** comes after C, so the health page shows every job, and every job already alerts.

- **F** waits for the trader.
- **The front door** can go live at any time; it changes nothing else.

3.3 What is done: steps 1–15

Merged in the code repository (pull requests #5 to #24) and, up to step 14, deployed to the box on 2026-10-08.

#	Step	Status	What is left, and where it goes
1	CI: every test on every pull request, against a real Postgres	**done**	—
2	one front door: the gateway, one login	**built**	the domain, TLS, closing the ports: any time (§3.10)
3	checks on save; start-up repairs run once	**done**	T6's photo time: the trader's decision (§3.13)
4	one trading calendar; real New York instants	**done**	—
5	the fact store: short interest, borrow, news	**in shadow**	switched on per family (D)
6	one market read per scan; hot sectors kept	**in shadow**	switched on (D)
7	the research worker: O'Neil facts with their dates; new tickers within minutes	**in shadow**	the whole chain in the worker (C); switched on (D)
8	run history; alerts on failed, missed, hung runs; the night	**partly**	one scheduler (B, C)
9	the registers copied to the shared record, all eleven tools imported with 0 differences; the discovery engine	**done**; the engine **off**	the engine retires with B (ARCHITECTURE §10)
10	the scorecard, the training set, an empty model slot	**prepared**	the scorecard page (E); the model (F)
11	the journal: one pairing engine for every source	**done, in part**	stops, the entry snapshot (D), the tables (JOURNAL.md)
12	qp: alerts on the strategy's own fill model, alert history; registers from the shared record (off)	**partly**	alert cycles from the desk (C); reads as of a day (D)
13	the verified backup, one-command restore, monthly rehearsal	**done**	the backup before every deploy (C); the analysis files as Parquet
14	one origin through the gateway	**partly**	phone first, ⓘ on every label (E)
15a	the tool context: one program can hold several tools, kept apart	**done**	—
15b	the comparison copy and its daily photo check, in its own program	**done**, not yet started on the box	becomes a context of the desk (B)

3.4 Step A: the desk starts, with the stopped tools

Changes:

- the **desk program** (`desk` under pm2): the web layer, the contexts, the desk's own port (DESK.md);
- each **stopped tool** opened read-only in its own context, its pages and its read routes served as the archive serves them today; a write is refused with the reason (STOPPED-TOOLS.md);
- the **photo-window guard** on the deploy, before the desk holds a running tool (BOX.md §4.2);
- the backup's **analysis files** read through the desk, each file opened read-only (BACKUPS.md).

The check. While the archive still runs, the desk serves each stopped tool under `/tools/T<n>/` on its own port. Every read route of every stopped tool is asked of both, over every register day, and must answer the same.

The switch. The archive stops. The desk listens on the stopped tools' ports, so every old address keeps working, and the gateway sends `/tools/T<n>/` to the desk.

Undo. Start the archive again.

Seen: the archive is gone; the stopped tools' pages are unchanged.

3.5 Step B: the desk takes the running tools

One tool at a time: T2, then T7, T11, T6, and **T1 last**. For each tool:

1. **Copy.** The tool is listed in `workerShadow`. The desk opens a **copy** context (DESK.md §3):
   - it follows the tool's screeners, settings, stars and today's shortlist, read-only from the tool's file every minute, and its schedules;
   - it starts from the tool's checkpoint when it starts mid-day;
   - it writes only `data/worker/<id>.db`: no shared-record rows, no alert, no desk job, no training.

   It starts only when the box has room. The first copy measures what a context costs, and the room needed is set from that.

2. **Compare the photo, daily.** Five minutes after the tool's photo, the two photos are compared:
   - the names: none missing, none extra;

- the screeners: each name found by the same screeners;
- every field, each put in one class:

Class	Means
`same`	equal
`market`	a number within 1 % (two reads of a quote seconds apart)
`time`	an instant within 2 minutes
`trader`	set by hand: `bias`, `inShortlist`
`different`	anything else

A clean day: the names and screeners match, and no field is `different`. The result goes to the phone as one line and is kept.

3. **Compare the routes.** Every read route of the tool is asked of the tool's own program and of the desk:
   - the routes that read the file must answer the same;
   - the routes that read the live list are compared with the photo's classes.
4. **Switch,** after **five clean trading days in a row**, on an evening, never within 15 minutes of a capture:
   - the tool is added to `deskTools`, and removed from `workerShadow`;
   - the deploy stops the tool's own program;
   - the desk opens the tool's file as its only writer, brings back today's list, registers its jobs and listens on its port;
   - the gateway sends `/tools/T<n>/` to the desk.
5. **Watch the next trading day:** the scans on their schedule, the photo at its minute, the outcomes at 16:05, every run record clean.

Undo, for one tool. Remove it from `deskTools` and deploy: the desk closes the tool's context, and its own program starts again on the same file. Nothing is lost, because the file is the same file.

T1, last, takes three things with it:

- **home:** the landing page, the lists across all tools and Manage tools move into the desk's own context (HOME.md);
- **the desk jobs** move into the desk's own context and its own file, `data/desk.db` (JOBS.md). Their past runs stay in T1's file, as history;
- **T1 becomes a tool like the others,** so it can be stopped. Home keeps port 3000, where the landing page has always been, and T1 takes port 3090 (WEB.md §3.1).

After the last tool:

- `deskTools` and `workerShadow` retire: the desk runs every running tool;
- the comparison programs and code (`desk-worker`, `src/worker/`), the discovery engine (off) and `deploy/tool-ctl.sh` retire;
- **T9, the benchmark,** stopped for lack of memory, runs again as a context. It is the screener every other must beat.

Seen: each move frees about 90 MB; one program to restart instead of five; T1 can be stopped; the benchmark is back.

3.6 Step C: one scheduler

Change	The check	Undo
The night. `scripts/retire-old-schedulers.sh` removes qp's systemd timer and the pm2 backup cron; the desk's night job runs the research, then the backup (JOBS.md)	the first week's nights recorded, each step ok, each backup clean	re-enable the timer and the cron
The whole O'Neil chain in the research worker (its steps 1–12); qp's nightly job no longer computes it (RESEARCH-WORKER.md)	for a week the night runs both, one after the other: the worker's facts must equal qp's outputs	qp's nightly chain again
qp's alert cycles started by the desk (`POST /api/alerts/cycle`), each minute 09:25–16:05 while alerts are on; qp's watcher thread not started (SIGNAL-ALERTS.md)	for a week the desk's cycles run beside the thread without writing history or notifying; their signals, per strategy, symbol and bar, must equal the thread's	qp's thread again
Signals on the phone, on their own topic (ALERTS.md §3.6)	a test signal reaches the phone	the page only
The journal's sync started by the desk (`POST /api/journal/sync`); the journal's own timer off (JOURNAL.md)	for a week both run; fills are keyed by their broker id, so the same fills arrive once	the journal's timer again
Every job alerts, frequent jobs after three failed runs in a row (ALERTS.md §3.1)	each kind of job made to fail once, on purpose, and seen to alert	—
The deploy backs up first (BACKUPS.md)	a deploy's backup in the bucket, before its restarts	—

Seen: qp's signals on the phone; one place for every job's runs.

3.7 Step D: switched on

Each switch alone, each when its check is clean, each undone by setting it back to shadow (SHARED-RECORD.md §3.6).

Switch	The check before	Then
`marketMode on`	a week of each tool's own read equal to the one read (`market_parity`); each tool's own hot-sector and bias thresholds equal to the desk's, or listed	every card is stamped from the one read; the R2 jobs retire, and R2 is read from the market history; then the corrections of MARKET.md
`factsMode on`, family by family	`fact_parity` clean for a week, and `source_calls` showing the tools share enough tickers for sharing to pay	the family is read from the store
`researchMode on`	the worker's facts equal qp's values, field by field, for a week; with qp stopped, the cards and the Market tab unchanged	the cards read the worker's facts; qp's O'Neil routes retire
`quantRegistersMode on`	past backtests re-run against the shared record give identical trades	qp reads every tool's registers from the shared record
qp reads facts and market states as of a day (to build)	the old days imported (§3.11); a sample of backtests with the CANSLIM gate compared	the CANSLIM control on qp's page (BACKTEST.md)
the journal's entry snapshot (to build)	facts as of a day available	each trade's snapshot (REVIEW.md)

Seen: fewer source calls; backtests gated on the market and CANSLIM for any day; each trade's snapshot.

3.8 Step E: pages and checks

Change	The check
the health page, `/api/status`, and its checks: sources, fact freshness, programs, the box, sign-ins (ALERTS.md)	each check made to fail once, on purpose (a wrong key, a stopped program), and seen on the page and the phone
the outside heartbeat	the desk stopped for 20 minutes outside the session: the alert arrives
the scorecard page per tool and screener (LEARNING.md)	each number against its API
live updates on open pages (PAGES.md)	a page left open follows the scans without a reload
phone first: every page readable at 430 px wide	every page at 430 px
ⓘ on every label, by tap (CARD.md)	every label has one

Undo. All of it is additive.

3.9 Step F: the model

When the trader chooses one (LEARNING.md):

- trained by the research worker, one per tool, from that tool's own history;
- **the check:** a walk-forward Brier score better than the tool's base rate;
- **undo:** the empty slot again.

3.10 Any time: the front door goes live

1. Fix `scripts/setup-domain.sh` for the box's system: it uses `apt-get`, and the box runs Amazon Linux 2023 (FRONT-DOOR.md).
2. The trader makes a free DuckDNS name and runs the script; Caddy takes a certificate.
3. In AWS, open 80 and 443, and close 3000–3120, 8765 and 8080 to the internet.

The check: a scan from outside finds only 80 (redirecting) and 443, and every page works through the one address. **Undo:** open the ports again.

3.11 The old data

What	Into	Status
every tool's R1, R3, shortlist, tool row and screeners, running and stopped	the shared record's copies	**done** (step 9a), 0 differences
the screeners' versions	each tool's file (`screener_versions`)	**to do** in B, from the shared record's versions, kept since step 9a
the old R2 market snapshots of every tool	`market_states`, marked imported, for the days before step 6	**to do** before qp reads market states as of a day (D)
the O'Neil files of each night, kept in the backups	facts as of that night	**to do** before qp's CANSLIM gate reads facts as of a day (D)
qp's strategies	`strategy_versions` in qp's file, each current strategy as version 1	**to do**, with strategy versions
the journal's trades	`fills`, `trades`, `trade_fills` in the journal's file	**to do**, with those tables; the Alpaca trades re-derived and compared

An old row that lacks a field the new tables record (a screener version, a market state, an O'Neil status frozen at the time) is imported with that field marked **unknown**, never guessed.

3.12 What stays exactly as it is

- **The research method:** one photo a day per tool; entries A and B; outcomes in ATR; the 1.3 good-move line; the benchmark; the mirrors.
- **The card:** every field, every Market-tab section, every source and its fallback order. Their home changes, their content does not.
- **Isolation:** each tool's results, model and history kept from every other.
- **A label, never a filter,** for every shared fact.
- **The O'Neil method as built;** qp's verified library and engine.
- **The one password and the nightly backup,** until what replaces each has proven itself.

3.13 Decisions still open

Decision	Needed by	Where
where the shared record lives: Railway, the box or AWS RDS	before D; to be taken up again on 2026-10-22	SHARED-RECORD.md §3.7
T6's photo (D-S9): after 11:30 with its entries, or its window opened earlier	any time; it does not block T6's move, which changes nothing T6 does	SCREENERS.md
the model	F	LEARNING.md

---

4. Contracts

Setting (`data/keys.json`)	Used in	Means
`workerShadow`	B	the tools run as comparison copies
`deskTools` (to build)	B	the tools the desk runs for real. The deploy starts no program of their own, and the gateway sends them to the desk. Retired after B
`factsMode`, `marketMode`, `researchMode`, `quantRegistersMode`	D	off, shadow or on, per area (SHARED-RECORD.md §3.6)
`discoveryMode`	—	off; retired in B

Comparison record	Where	Step
the photo comparison of each copy	`worker_parity`, in `data/worker/<id>.db`; `scripts/worker-report.js`	B
the register copies against each tool's file	the daily comparison; `GET /api/registers/parity`	done
the facts against the tools' own	`fact_parity`; `scripts/facts-report.js`	D
the market read against each tool's own	`market_parity`	D

---

5. When it fails

Event	What happens
a comparison is not clean	the step waits. The difference is explained (a fault the new part fixes) or fixed, and the clean week starts again
a moved tool misbehaves	its undo: its own program again, on the same file
the box runs short of memory	the copy does not start, and says why. The next move frees the room
a step's change reaches a page the trader uses	it is undone first, then fixed

---

6. Built on

- the first migration plan, steps 1–20, which this replaces from step 15 on: its steps 15 and 16 (two programs, desk-worker and desk-web) became A and B (one program), its 17 became D, its 18 and 19 became E, its 20 became F;
- the earlier rebuild draft;
- the known issues of the old desk: screeners, qp, journal;
- the code repository's `docs/WORKER.md` (the comparison copy) and `docs/SETUP-NOW.md` (the commands on the box).

What steps 1–15 taught:

1. **Measure a saving before building for it.** The discovery engine merges only identical requests. Every tool's screeners differ, so it saves no call, and while it compares it doubles them.
2. **Fact sharing depends on overlap.** `source_calls` counts the requests against the real calls, family by family, and decides each switch.
3. **Shadow first was right.** The import found no difference, and the first deploy changed nothing the trader uses. But it showed the trader little, so each step of the program half ends with something visible.
4. **Two programs were the hardest part of the earlier plan.** Everything a page shows from the scans would have had to be copied into the database all day. One program removes that (DESK.md).

7. Status

Step	Status
1–15	**done** or in shadow (§3.3)
A	**to build**: the desk program is the next build
B	**to build**; the comparison copy exists as its own program
C	**to build**; the night job and its handover script exist
D	**to do**, switch by switch
E	**to build**
F	**waiting** for the trader
the front door live	**ready**, once `setup-domain.sh` is fixed